AI ENGINEERING COURSE

Module 5

Modern LLM Architecture

7 lessons · Personal offline study copy

Personal offline study copy. Content © Outcome School / Amit Shekhar. Not for redistribution. See study/NOTICE.md.

Contents

Module 5: Modern LLM Architecture

- 5.1 Evolution of LLM Architecture
- 5.2 What is Mixture of Experts (MoE) and How Does It Work?
- 5.3 What is Grouped Query Attention (GQA) and Why Do LLMs Use It?
- 5.4 How does Sliding Window Attention work?
- 5.5 How do Attention Sinks work?
- 5.6 What is Flash Attention and Why Is It So Fast?
- 5.7 What is DeepSeek-V4 and How Does It Work? Architecture Explained

MODULE 5

Modern LLM Architecture

7 lessons · ~1 min overview

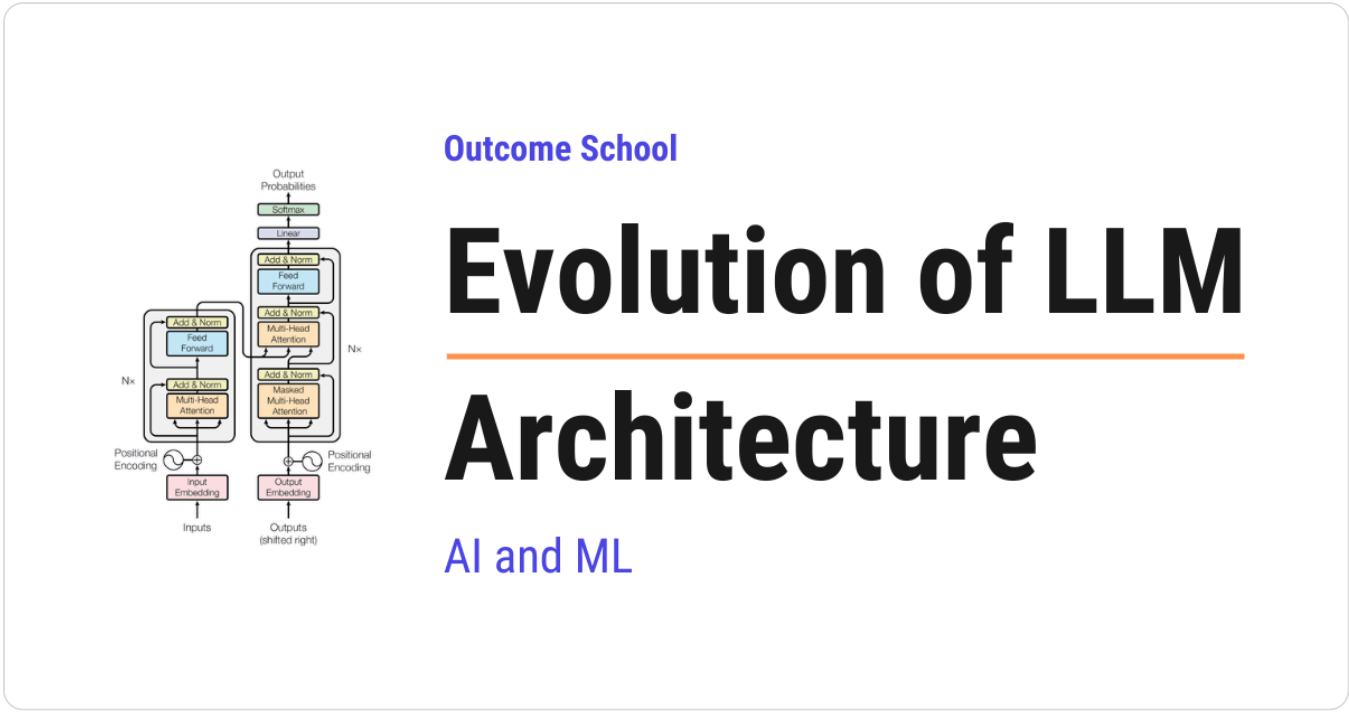
In this module, we will learn the improvements that modern LLMs add on top of the basic Transformer to become bigger, faster, and able to handle longer inputs. At the end, we will see all of these ideas together inside a real model.

By the end of this module, we will be able to read the architecture section of any new open-weight model and understand every design choice in it.

LESSON 5.1

Evolution of LLM Architecture

Lesson 5.1 · ~30 min · Published September 21, 2026 · https://outcomeschool.com/blog/evolution-of-llm-architecture



LLM architecture evolved in stages: from RNNs that read one word at a time, to Attention, to the Transformer, to making models bigger, and then to Mixture of Experts (MoE) that makes big models cheaper to run.

In this blog, we will learn about the Evolution of LLM Architecture, the step-by-step journey of how the design of large language models changed from simple word-by-word readers to the massive AI models we use today. We will also see why the early models kept forgetting, how attention solved that problem, how the Transformer removed the slow parts, how making models bigger made them smarter, how Mixture of Experts made big models cheaper to run, what problems still remain, and what is coming next.

We will cover the following:

- What is an LLM Architecture?
- Stage 1: Reading one word at a time (RNN)
- Stage 2: Attention
- Stage 3: The Transformer
- Stage 4: Scaling
- Stage 5: Mixture of Experts (MoE)
- Stage 6: New Directions
- Summary of the evolution

Let's get started.

What is an LLM Architecture?

An LLM Architecture is the blueprint of a large language model. It describes how the model reads text, how it remembers what it has read, and how it produces the next word.

Let's break the name.

LLM = Large + Language + Model

- **Language**: it works with text, the words we read and write every day.
- **Model**: it is a computer program that has learned patterns from a huge amount of text.
- **Large**: it has learned from a very large amount of text and has a very large number of internal settings.

Architecture is simply the design. Let's say we are building a house. The architecture is the plan that tells us where the rooms go, how big the doors are, and how the water flows. The bricks and cement are the same for every house, but the plan decides how good the house is.

Similarly, in an LLM, the numbers and the math are similar across models, but the architecture decides how the model reads, remembers, and answers.

Every big step in LLM architecture happened because the previous design had a problem.

- The first models forgot what they read.
- Then attention solved forgetting, but the models were still slow.
- Then the Transformer made them fast, but nobody knew how big to make them.
- Then scaling made them smart, but too costly to run.
- Then Mixture of Experts made them cheaper.
- And the story is still going on.

So, the best way to understand today's LLM is to walk through this timeline, one problem and one solution at a time.

Do not worry, we will learn about each of them in detail.

Stage 1: Reading one word at a time (RNN)

Before Transformers, the most popular design for reading text was the **Recurrent Neural Network (RNN)**.

A neural network is a computer program made of a very large number of connected numbers that learns patterns from examples. Showing it a lot of examples so that it learns from them is called

training. Do not worry about the internal details. For this blog, we only need to know how each design reads text.

Note: A model does not read whole words. It first cuts the text into small pieces called **tokens**. A token can be a full word, a part of a word, or a punctuation mark. For the sake of understanding, we will say word everywhere in this blog, but wherever we say word, the model is really working with a token.

Suppose we give an RNN the sentence: "The cat sat on the mat."

An RNN reads it one word at a time, from left to right. After reading each word, it updates a single small memory and then moves to the next word. The memory is like a tiny note. After reading "The", the note has something about "The". After "cat", the note is rewritten to remember "The cat". And so on, till the end.

At the end, the whole sentence is squeezed into that one small note.

Let's understand this with a picture:

```
Word:     The       cat       sat       on        the       mat
           |         |         |         |         |         |
           v         v         v         v         v         v
Note:   [note] -> [note] -> [note] -> [note] -> [note] -> [note]
                                                            |
                                                            v
                                                     the whole sentence
                                                     lives in this one note
```

Here, there is only one note. The arrow from one note to the next shows that the same note is rewritten after every word. The only thing carried forward from the past is this note, nothing else.

This worked for short sentences. But, here is the catch.

Problem 1: It forgets.

Let's say the text is very long, like a full paragraph. By the time the model reaches the end, the note has been rewritten so many times that the early words are almost gone. If the paragraph starts with "Riya went to the market" and ends with "and then she bought", the model struggles to remember that "she" means Riya.

```
Riya  went  to  the  market  and  then  she  bought
 |                                       |
 |   the note is rewritten again and     |
 |   again between these two words       |
 v                                       v
"Riya" is written into       by now "Riya" has mostly
the note at the very start   faded out of the note
```

Here, we can see that "Riya" entered the note at the start, but every word after it overwrote the note a little more. By the time the model reaches "she", very less of "Riya" is left.

Problem 2: It is slow to train.

The model must read word 1 before word 2, and word 2 before word 3. Means, it cannot read all the words at the same time. So, even if we have a very powerful computer that can do thousands of things in parallel, the RNN uses it one step at a time.

Improved versions of RNN, called LSTM and GRU, reduced the forgetting a lot, but they did not remove it, and the slow one-by-one reading remained.

The issue with this approach is forgetting and slow training. Let's see how the next approach solve this issue.

Stage 2: Attention

Here comes the **Attention** into the picture.

Attention is a mechanism that lets the model look back at every word in the input and decide how much to focus on each one while processing the current word.

In simple words, instead of keeping one tiny note, we keep the whole page open in front of us. When we need to understand a word, we look at whichever earlier words matter for it.

Let's see the difference with a picture:

```
RNN:
  Riya -> went -> to -> the -> market -> and -> then -> she
  (one small note passed along the chain, rewritten at each step)
Attention:
  Riya   went   to   the   market   and   then   she
   ^      ^      ^    ^      ^       ^      ^     |
   |      |      |    |      |       |      |     |
   +------+------+----+------+-------+------+-----+
            "she" looks directly at every earlier word
```

Here, in the RNN, "she" can only see the note that reached it through the chain. In attention, "she" has a direct line to every earlier word. Nothing is lost on the way.

Attention = Look at every word + Decide how much each one matters + Mix them accordingly

Let's take the example again: "Riya went to the market and then she bought mangoes."

When the model is processing the word "she", attention lets it look at all the words before it and ask: which words help me understand "she"? The answer is "Riya". So, the model gives high attention to "Riya" and low attention to words like "to" and "the".

How attention decides where to focus

The best way to learn this is by taking an example.

Suppose the model is processing "she". For the sake of understanding, let's look at only 4 of the earlier words. Attention gives each word a score that tells how important it is. Then it converts those scores into weights that add up to 1.

- Riya: 0.70
- went: 0.10
- market: 0.15
- then: 0.05

Here, we can see that "Riya" gets 70 percent of the focus. The model then builds its understanding of "she" mostly from "Riya", a little from "market", and very less from the others.

Let's see how the model builds "she" from these weights:

```
0.70 x Riya    ----+
0.10 x went    ----+
0.15 x market  ----+----> understanding of "she"
0.05 x then    ----+
----
1.00  (the weights add up to 1)
```

Here, each earlier word is multiplied by its weight and then all of them are added together. Since "Riya" has the biggest weight, the result is mostly "Riya".

That is all attention is. **A weighted mix of the other words, where the weights say how much each word matters.**

This simple idea is the heart of every modern LLM.

Query, Key, and Value

Inside attention, every word plays three roles. Let's take an analogy of a library.

- **Query**: the question we are asking. When we are at "she", the query is "who is this person?"
- **Key**: the label on every book. Each earlier word carries a label that says what it is about. The word "Riya" carries a label like "a person's name".
- **Value**: the content inside the book. The actual information that the word carries.

The model compares the query with every key. The keys that match well get high scores. Then it collects the values, weighted by those scores. That gives us the weighted mix we saw above.

Here is how the flow looks:

```
Query from "she": who is this person?
         |
         v
Compare the query with every key:
  Key of "Riya"   : a person's name  -> strong match
  Key of "went"   : an action        -> weak match
  Key of "market" : a place          -> weak match
         |
         v
Collect the values, weighted by the match
         |
         v
Understanding of "she", built mostly from the value of "Riya"
```

Here, the query is the question, the keys decide which words answer it well, and the values are the information we finally collect. The key only helps in matching. The actual content comes from the value.

We have a detailed blog on the math behind Q, K, and V that explains this step by step with real numbers.

Self-Attention

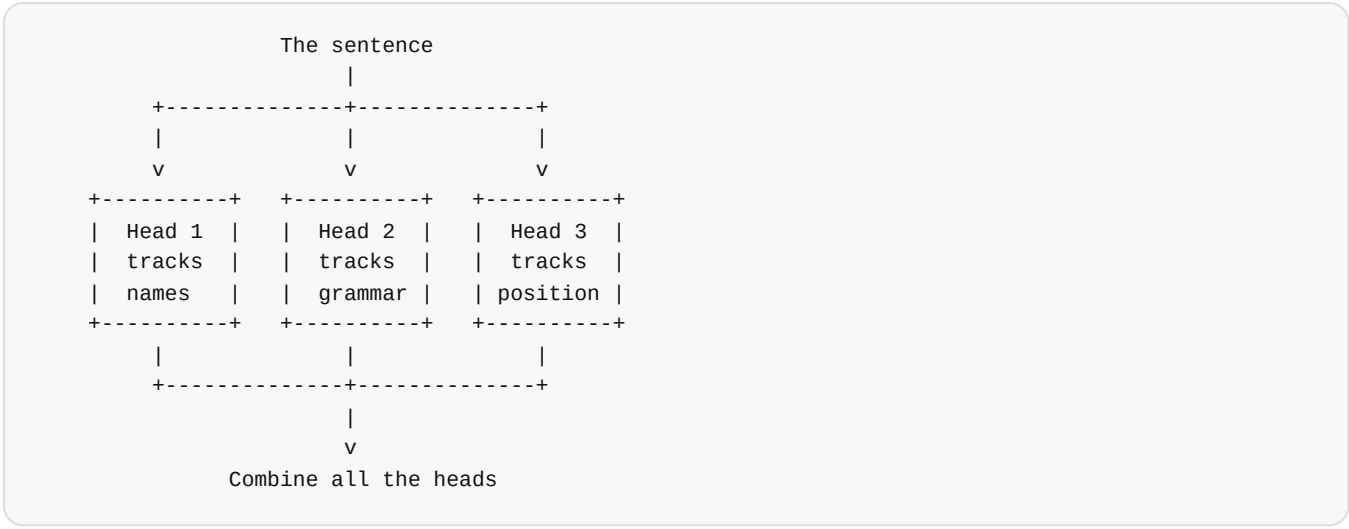
When the words of a sentence pay attention to other words of the same sentence, we call it as **Self-Attention**. The sentence is looking at itself. This is the form of attention used inside LLMs.

Multi-Head Attention

One attention can focus on one kind of relationship at a time. But a sentence has many relationships. "she" relates to "Riya" by meaning, "bought" relates to "mangoes" by grammar, and so on.

So, instead of one attention, we run several attentions side by side. Each one is called a **head**. One head learns to track names, another learns to track grammar, another learns to track position, and etc. Then we combine what all the heads found.

This is called **Multi-Head Attention**. It is like having a team of readers where each reader is looking for a different thing in the same sentence.



Here, all the heads read the same sentence at the same time, but each head is looking for a different kind of relationship. At the end, we combine what all of them found into one result.

Attention solved the forgetting problem, because now no word in the input is ever squeezed out of memory. Every word can be looked at directly.

But, in the beginning, attention was added on top of RNNs. The RNN still read the input one word at a time, and attention only helped the model look back at what the RNN had already read. So, the one-by-one slow reading was still there.

The issue with this approach is that the model still read slowly, one word at a time. Let's see how the next approach solve this issue.

Stage 3: The Transformer

In 2017, a research paper with the title "Attention Is All You Need" asked a bold question: if attention is doing the real work, why keep the RNN at all?

So, they removed the RNN completely and built a model only from attention and a few simple pieces of math. That model is called the **Transformer**.

A Transformer is a model that reads all the words of the input at the same time and uses attention to understand how they relate to each other.

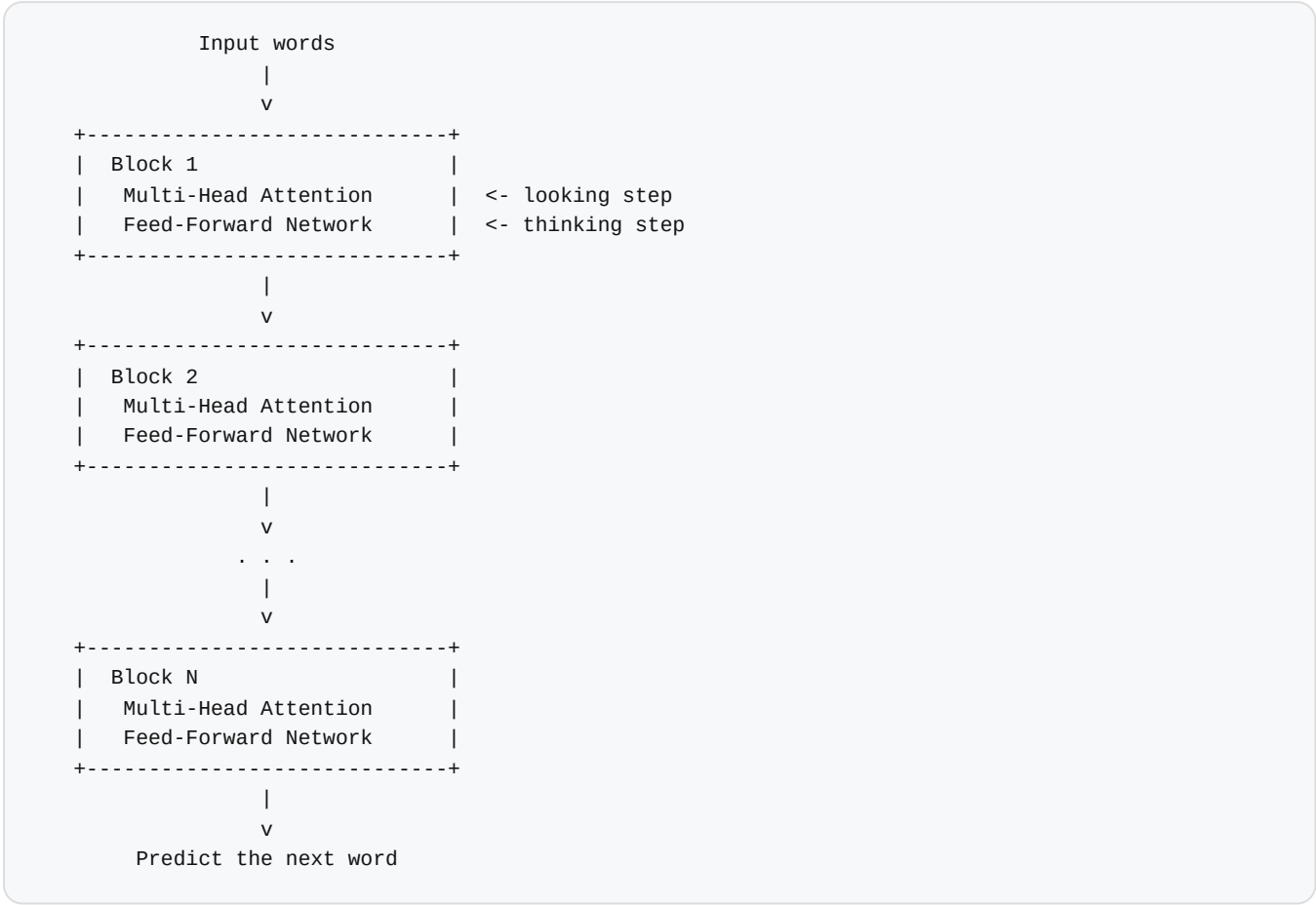
What is inside a Transformer

A Transformer is made of blocks stacked on top of each other. Each block has two main parts:

- **Multi-Head Attention**: the words look at each other and gather information. This is the looking step.
- **Feed-Forward Network**: a simple layer of math that each word passes through on its own to process what it gathered. This is the thinking step.

We stack many such blocks. Early blocks learn simple things like grammar. Deeper blocks learn bigger things like meaning and reasoning. A small model has around 12 blocks. A large model has 100 or more.

Here is how the blocks are stacked:



Here, the words enter at the top, pass through every block one after another, and the last block predicts the next word. Every block does the same two steps: look, then think.

The word order problem

There is one interesting problem. Since the Transformer reads all the words at once, it does not naturally know which word came first. "Dog bites man" and "Man bites dog" would look the same to it.

So, here comes the **Positional Encoding** to the rescue. Before the words enter the model, we add a small signal to each word that says its position: word 1, word 2, word 3, and so on. Now, the model knows the order.

The original Transformer used a fixed signal for this. Modern LLMs use newer ways to do the same job, like **Rotary Position Embedding (RoPE)**, which is used in LLaMA and many other models. The method changed, but the idea is the same: tell the model where each word sits.

Encoder, Decoder, and Decoder-only

The original Transformer had two halves.

- **Encoder**: reads the input and understands it.
- **Decoder**: writes the output one word at a time, looking at what the encoder understood and at what it has already written.

This was built for translation. Read English, write French.

Later, researchers found that for generating text, we only need the decoder half. Give it some words, and it predicts the next word. Then add that word to the input, and predict the next. This is the **Decoder-only Transformer**, and it is the design used by GPT and almost every chat model today.

```
Encoder-Decoder (built for translation):
  "The cat sat"  --->  [ Encoder ]  --->  [ Decoder ]  --->  "Le chat s'est assis"
Decoder-only (used by chat models):
  "The capital of India is"      --->  [ Decoder ]  --->  "New"
  "The capital of India is New"  --->  [ Decoder ]  --->  "Delhi"
```

Here, the decoder-only model has a single stream. The word it produces is added back to the input, and it predicts again. This loop continues till the answer is complete.

This loop is called autoregressive generation, and we have a detailed blog on Autoregressive Models that explains how it works.

Why the Transformer won

The Transformer processes all the words in parallel. This matched perfectly with GPUs, which are chips built to do thousands of small calculations at the same time. Training that took weeks on RNNs took days on Transformers.

```
RNN (one word per step):
  Step 1    Step 2    Step 3    Step 4    Step 5    Step 6
   The   ->  cat   ->  sat   ->  on    ->  the   ->  mat
Transformer (all words in one step):
  Step 1
   The     cat     sat     on      the     mat
    |       |       |       |       |       |
    +-------+-------+-------+-------+-------+
                all processed together
```

Here, the RNN needs 6 steps for 6 words, and each step must wait for the previous one to finish. The Transformer handles all 6 words in a single step, so all the GPU cores stay busy at the same time.

We have a detailed blog on how RNNs and Transformers differ that covers this comparison end to end.

Note: This parallel reading applies to the input and to training. While writing the answer, the model still produces one word at a time, as we saw in the decoder-only loop above. Reading is parallel,

writing is one by one. Keep this in mind, it will matter when we reach the KV Cache later.

And that speed opened the door to the next stage. If training is fast, why not train a bigger model on more data?

This was all about the Transformer. It solved slow training. Now, the next big question is: how big do we make it?

Stage 4: Scaling

Scaling means making the model bigger in three ways: more parameters, more training data, and more compute.

Let's understand each.

- **Parameters**: the internal settings of the model. Let's say a model is a giant board with billions of tiny knobs. Training turns each knob a little to make the predictions better. The number of knobs is the number of parameters.
- **Data**: the text the model learns from. Books, websites, articles, code, and etc.
- **Compute**: the total amount of calculation done during training. More compute means more GPUs running for more time.

The surprising discovery

Around 2020, researchers noticed something surprising. When they increased parameters, data, and compute together, the model got better in a smooth and predictable way. There was no sudden wall. Bigger kept getting better.

These are called **Scaling Laws**. They gave a formula that said: if we spend this much compute, the model will reach roughly this much quality.

Hence, models grew very fast.

- GPT-2 had about 1.5 billion parameters.
- GPT-3 had about 175 billion parameters, more than 100 times bigger, just one year later.

And GPT-3 did things that GPT-2 could not. It could write essays, answer questions, and even write code, that too without being trained for those specific tasks. These new abilities that appear only at a large size are called **emergent abilities**.

Note: Some researchers argue that these abilities do not appear suddenly. They grow smoothly with size and only look sudden because of how we measure them. The term is still widely used, so it is good to know it.

Balancing data and size

Then came a correction. In 2022, a research paper called Chinchilla found that many big models were undertrained. They had too many parameters and too little data.

The lesson was simple: **for the best result, grow the data along with the parameters.** A smaller model trained on much more data can beat a bigger model trained on less data. This is why later models like LLaMA were trained on trillions of tokens.

The cost problem

Scaling worked, but it created a new problem.

In a normal Transformer, every parameter is used for every single word. If a model has 175 billion parameters and we ask it a question of 100 words, all 175 billion parameters do work for each of those 100 words. This is called a **dense** model, because everything is active all the time.

```
Dense model:
  word 1    --->  all 175 billion parameters work  --->  output
  word 2    --->  all 175 billion parameters work  --->  output
  word 3    --->  all 175 billion parameters work  --->  output
   ...
  word 100  --->  all 175 billion parameters work  --->  output
```

Here, whether the word is a simple "the" or a difficult technical term, the full model works on it. There is no way to do less work for an easy word.

So, doubling the size of the model doubles the cost of every answer. Training became very costly, and running the model for millions of users became even more costly.

The issue with this approach is that every increase in size brings the same increase in running cost. Let's see how the next approach solve this issue.

Stage 5: Mixture of Experts (MoE)

Here comes the **Mixture of Experts (MoE)** to the rescue.

Mixture of Experts is an architecture where the model is divided into many smaller parts called experts, and for each word, only a few experts are used.

Let's decompose it.

Mixture of Experts = Many Experts + A Router that picks a few of them

Let's say we go to a big hospital. The hospital has a heart specialist, a bone specialist, a skin specialist, and an eye specialist. When we walk in, the receptionist listens to our problem and sends us to one or two relevant doctors. All four doctors do not examine every patient.

- The doctors are the **experts**.
- The receptionist is the **router**, also called the gate.
- Sending each patient to only a few doctors is called **sparse activation**.

How MoE works inside the Transformer

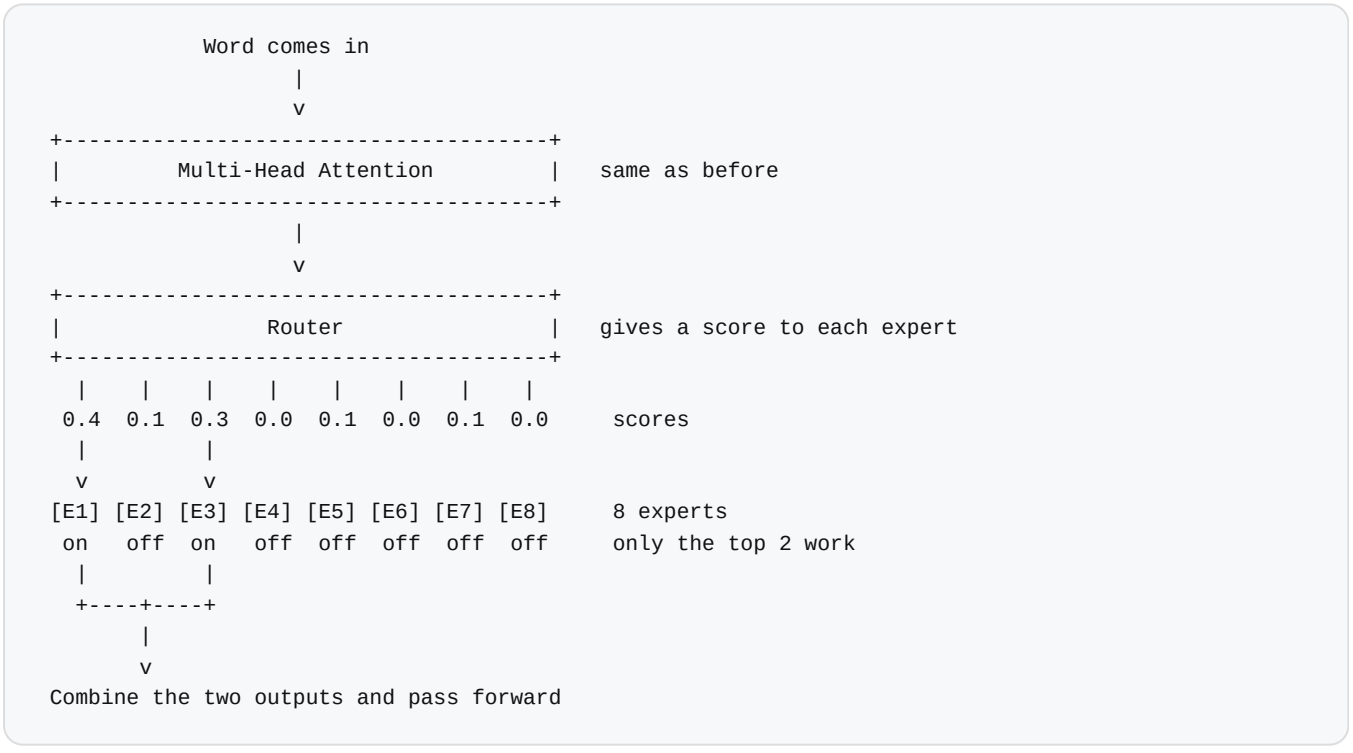
Remember the Feed-Forward Network inside every Transformer block, the thinking step after attention? In an MoE model, that single feed-forward network is replaced by many feed-forward networks, the experts. A small router sits in front of them.

For every word:

- The router looks at the word and gives a score to each expert.
- It picks the top few experts, usually the top 1 or top 2.
- Only those experts process the word.
- Their outputs are combined and passed forward.

The attention part stays the same. Only the thinking step becomes a team of specialists.

Let's understand this with a picture:



Here, the router gave the highest scores to Expert 1 and Expert 3, so only those two do the work for this word. The other six sit idle for this word. For the next word, the router can pick a different pair.

Total parameters vs active parameters

This is the key idea, so let's take numbers.

Suppose we start with a model of about 7 billion parameters and, in each block, replace its single Feed-Forward Network (the thinking step) with 8 expert Feed-Forward Networks. This is the basic design of Mixtral 8x7B. The name means 8 experts built on the architecture of a 7 billion base model. The model does not become 8 x 7 = 56 billion, because the attention part and the other shared pieces are not copied. Only the Feed-Forward Network is copied 8 times. Together with the shared parts, the model has about 47 billion parameters in total.

Note: The numbers here are rounded for the sake of understanding.

Now, for each word, the router picks only 2 of the 8 experts. So, the work done for each word uses about 13 billion parameters.

- **Total parameters**: about 47 billion. This is the knowledge the model holds.
- **Active parameters**: about 13 billion. This is the work done per word.

```
Total parameters (what the model knows):
  [ shared parts ] [E1] [E2] [E3] [E4] [E5] [E6] [E7] [E8]   = about 47 billion
Active parameters (what works for one word):
  [ shared parts ] [E1]      [E3]                            = about 13 billion
```

Here, all 8 experts live inside the model, but for any one word, only 2 of them wake up along with the shared parts.

Means, the model has the knowledge of a 47 billion model but a running cost close to that of a 13 billion model. It is not exactly equal, because the router and the shared parts also do some work, but it is far cheaper than running all 47 billion. That's the beauty of MoE.

This way we can use MoE to solve the interesting problem of cost without giving up knowledge.

Bigger models like DeepSeek-V3 push this much further. It has about 671 billion total parameters, but only about 37 billion are active for each word.

What each expert learns

One more thing to notice: we do not tell the experts what to specialize in. During training, the router and the experts learn together. Some experts end up handling punctuation, some handle numbers, some handle parts of code, and some handle a particular kind of word. The split is not always clean, but it happens on its own.

The problems with MoE

MoE is not free. It has its own issues.

- **Memory**: even though only 2 experts work per word, all 8 must be loaded in memory, because the next word can go to a different pair. So, MoE saves compute but not memory.
- **Load balancing**: if the router keeps sending everything to the same two experts, the other experts never learn. So, we add a small extra rule during training that pushes the router to spread the

work. Without this, MoE fails.

- **Training complexity**: routing decisions make training less stable and harder to get right.

Still, MoE is the reason we can have models with hundreds of billions of parameters that respond in reasonable time. Most of the biggest models today use MoE in some form.

This was all about Mixture of Experts. It solved the cost of the thinking step. But the attention step has its own cost, and that brings us to what is happening now.

Stage 6: New Directions

Researchers are now working in a few directions at the same time. Let's look at the important ones.

The long context problem

The text we give to the model at one time is called the **context**. Today, we want to give it whole books, long conversations, and big codebases. That creates a problem.

Attention compares every word with every other word. If the input has 1,000 words, that is 1,000 x 1,000 = 1 million comparisons. If the input has 100,000 words, that is 10 billion comparisons. Doubling the input makes these comparisons four times costlier.

```
4 words  ->  4 x 4 = 16 comparisons
         The   cat   sat   on
  The     x     x     x     x
  cat     x     x     x     x
  sat     x     x     x     x
  on      x     x     x     x
8 words  ->  8 x 8 = 64 comparisons   (2 times the words, 4 times the cost)
```

Here, every row is a word looking at every column. Adding one word adds a full new row and a full new column, so the grid grows much faster than the input.

Also, while generating the answer, the model stores the keys and values of every earlier word so that it does not calculate them again. This storage is called the **KV Cache**. For long inputs, the KV Cache becomes huge and eats up memory.

A few designs came to fix this:

- **Grouped Query Attention (GQA)**: many attention heads share the same keys and values instead of each head having its own. This shrinks the KV Cache a lot with very less loss in quality. Used in LLaMA 3 and many other models.
- **Multi-Head Latent Attention (MLA)**: compresses the keys and values into a small form before storing them, and expands them back when needed. Used in DeepSeek models.
- **Sliding Window Attention**: each word looks only at a fixed window of nearby words instead of everything. Information still travels far by passing through many blocks. Used in Mistral.

Let's see each of them with a picture. First, GQA:

```
Multi-Head Attention:                Grouped Query Attention:
  Head 1 -> own Key + Value            Head 1 --+
  Head 2 -> own Key + Value            Head 2 --+--> shared Key + Value (group A)
  Head 3 -> own Key + Value            Head 3 --+
  Head 4 -> own Key + Value            Head 4 --+
  Head 5 -> own Key + Value            Head 5 --+
  Head 6 -> own Key + Value            Head 6 --+--> shared Key + Value (group B)
  Head 7 -> own Key + Value            Head 7 --+
  Head 8 -> own Key + Value            Head 8 --+
  8 sets stored in the KV Cache        2 sets stored in the KV Cache
```

Here, the heads are divided into groups, and all the heads in one group share one set of keys and values. With 8 heads in 2 groups, the KV Cache becomes four times smaller. Each head still asks its own query, so the quality stays almost the same.

Now, let's see MLA:

```
Normal attention:
  Key + Value (big)  ------------------------------>  stored in KV Cache (big)
Multi-Head Latent Attention:
  Key + Value (big)  --> compress -->  small form  -->  stored in KV Cache (small)
                                              |
                                              v
                          expand back to full size only when needed
```

Here, instead of storing the big keys and values, MLA stores a small compressed form. When the model needs them, it expands the small form back. So, the KV Cache holds much less, and the model still gets the full information when it looks back.

Finally, let's see Sliding Window Attention:

```
Full attention:
  w1   w2   w3   w4   w5   w6   w7   w8
   ^    ^    ^    ^    ^    ^    ^    |
   +----+----+----+----+----+----+----+
         w8 looks at all 7 earlier words
Sliding Window Attention (window = 3):
  w1   w2   w3   w4   w5   w6   w7   w8
                       ^    ^    ^    |
                       +----+----+----+
         w8 looks only at the 3 nearest words
How information still travels far:
  Block 1:  w5 gathers from w2, w3, w4
  Block 2:  w8 gathers from w5, w6, w7, and w5 already carries w2, w3, w4
```

Here, in Block 1, w5 has already gathered information from w2, w3, and w4. In Block 2, when w8 looks at w5, it indirectly gets that information too. So, even with a small window, information from far away still reaches, one block at a time.

Because of these ideas, models moved from reading about 2,000 tokens at a time to reading a million tokens at a time.

A new kind of memory: State Space Models

Some researchers went back to the idea of reading one word at a time, but with a much smarter memory. These are called **State Space Models**, and the popular one is **Mamba**.

In simple words, Mamba keeps a running memory like an RNN, so its cost grows in a straight line with the input length instead of squaring. But unlike the old RNN, it looks at each incoming word and decides what to keep and what to throw away from memory. So, important information from far back survives much better than it did in the old RNN.

Here is a rough picture of how the cost grows. The numbers are only for comparison, not real measurements:

```
Input length:      1,000     2,000     4,000     8,000  words
Attention cost:    1 unit    4 units   16 units  64 units   (squares)
Mamba cost:        1 unit    2 units   4 units   8 units    (straight line)
```

Here, every time the input doubles, the attention cost becomes 4 times, but the Mamba cost only becomes 2 times. For very long inputs, this difference becomes huge.

Today, we also see **hybrid** models that mix a few attention blocks with many Mamba blocks, in order to get the accuracy of attention with the speed of Mamba.

One model for text, images, and audio

Early LLMs read only text. Now, the same Transformer design is being used for images, audio, and video. An image is cut into small patches, each patch is turned into a token just like a word, and the same attention handles it. Audio and video are handled in a similar way. These are called **multimodal** models.

Thinking longer instead of growing bigger

For years, the way to a smarter model was a bigger model. Now, there is a second way. Let the model think for longer before answering.

Reasoning models generate a long chain of thinking steps before giving the final answer. In those steps, they often try different paths and check their own work. This is called scaling at **inference time**. Inference is just the word for the time when the model is answering. Means, we spend more compute when answering instead of only when training.

So, the scaling story now has two directions: scale the training, and scale the thinking.

Summary of the evolution

Here is the whole journey in one picture:

```
RNN  --->  Attention  --->  Transformer  --->  Scaling  --->  MoE  --->  New Directions
forgets    fixes            removes the        makes the      cuts the   long context,
and slow   forgetting       slow RNN           model smart    cost       Mamba, reasoning
```

Here, every arrow is a problem being solved. Each stage kept what worked in the previous stage and fixed the biggest thing that did not.

Let me tabulate the whole journey for your better understanding.

Stage	Key idea	Problem it solved	Problem it left behind
RNN	Read one word at a time with a single memory	Could read text of any length	Forgot early words, slow to train
Attention	Look at every earlier word with weights	Forgetting	Still slow with RNN underneath
Transformer	Only attention, all words in parallel	Slow training	How big to make it
Scaling	More parameters, data, and compute	Model quality	Cost grows with size
Mixture of Experts	Use only a few experts per word	Running cost of large models	Memory, load balancing
New Directions	GQA, MLA, Mamba, multimodal, reasoning	Long context, new inputs, smarter answers	Still evolving

Here, we can see the pattern clearly. Every stage kept the good part of the previous stage and fixed its biggest problem.

- Attention kept the idea of reading text and fixed forgetting.
- Transformer kept attention and removed the slow part.
- Scaling kept the Transformer and made it bigger.
- MoE kept the size and cut the cost.
- The newer ideas keep all of it and push further, giving us longer context, new kinds of input, and smarter answers.

Now we must have understood the evolution of LLM architecture, from attention to scaling to Mixture of Experts and beyond. The next time we hear about a new model, we can ask a simple question: which problem from this timeline is it trying to solve?

Frequently Asked Questions

Does a Mixture of Experts model use less memory than a dense model?

No. MoE saves compute but not memory. Even though only a few experts work for each word, all the experts must be loaded in memory, because the next word can go to a different pair. So the running cost per word drops, but the full model still has to sit in memory.

Is Mixtral 8x7B a 56 billion parameter model?

No. Mixtral 8x7B has about 47 billion parameters in total, not 56 billion. Only the Feed-Forward Network is copied 8 times, while the attention part and the other shared pieces are not copied. For each word, only 2 of the 8 experts are used, so about 13 billion parameters are active.

Is a bigger model always better than a smaller one?

No. The Chinchilla paper found that many big models were undertrained, with too many parameters and too little data. A smaller model trained on much more data can beat a bigger model trained on less data. So, for the best result, we should grow the data along with the parameters.

Does a Transformer write the whole answer in parallel?

No. The Transformer reads the input in parallel, and training is parallel too. But while writing the answer, the model still produces one word at a time. Each new word is added back to the input, and the model predicts again until the answer is complete.

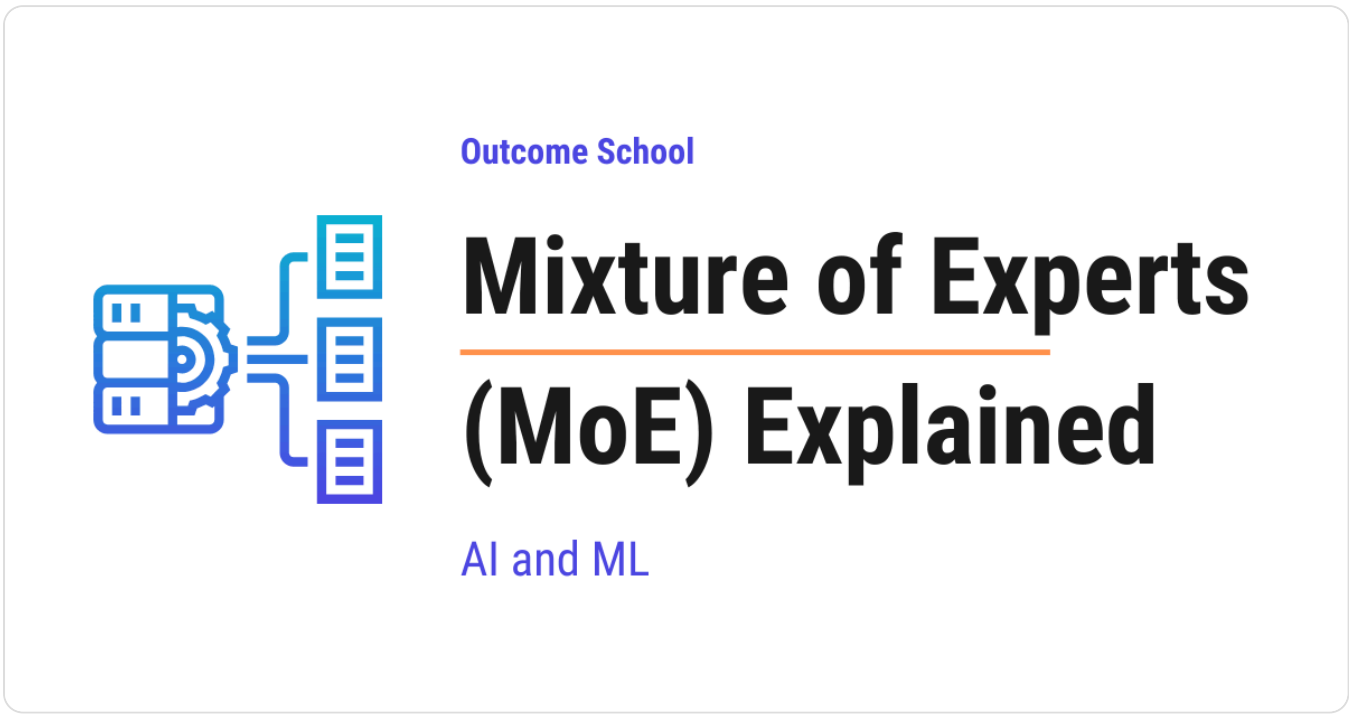
Is Mamba just an RNN?

No. Mamba keeps a running memory like an RNN, so its cost grows in a straight line with the input length. But unlike the old RNN, it looks at each incoming word and decides what to keep and what to throw away. So important information from far back survives much better.

LESSON 5.2

What is Mixture of Experts (MoE) and How Does It Work?

Lesson 5.2 · ~14 min · Published April 9, 2026 · https://outcomeschool.com/blog/mixture-of-experts



Mixture of Experts (MoE) is a neural network architecture that has many smaller networks called experts, and a router that activates only a few of them for each input. This makes large models faster and cheaper to run.

In this blog, we will learn about the Mixture of Experts (MoE) architecture - understanding what experts are, how the router picks them, why MoE makes large models faster and cheaper, and why it powers many of today's most powerful Large Language Models (LLMs).

When we hear "Mixture of Experts", it sounds complex. But do not worry. If we break it down into its individual parts, every single piece is simple. Our goal is to explain this architecture so clearly that by the end, we will be able to explain how Mixture of Experts works to anyone.

We will cover the following:

- Why Mixture of Experts was needed
- What an "expert" really means
- The router and how it picks experts
- Where MoE sits inside a Transformer
- Sparse activation and why it saves compute
- Load balancing across experts

- Advantages and challenges of MoE
- Why MoE powers many modern LLMs

Let's get started.

The Big Picture

Before we go into the details, let's understand the big picture.

A **Mixture of Experts** model is a special kind of neural network where, instead of using one big network for every input, the model has **many smaller networks called experts** and only activates **a few of them** for each input. A small component called the **router** decides which experts to use for each input.

In simple words: **Mixture of Experts = Many small expert networks + A router that picks which ones to use.**

Think of it like a hospital. A hospital does not have one single doctor who treats every patient. It has many specialists - a heart specialist, a skin specialist, a brain specialist, and so on. When a patient walks in, a receptionist (the router) sends them to the right specialist (the expert). The patient does not need to meet every doctor in the hospital. They only meet the ones who are relevant to their problem.

This is exactly how Mixture of Experts works inside an LLM.

Why Was Mixture of Experts Needed?

Before MoE, large language models were **dense**. Dense means every token uses every single parameter of the model. If a model has 100 billion parameters, every token uses all 100 billion parameters during inference.

This had two big problems:

Problem 1: Compute cost. As models grew bigger to become smarter, the compute cost grew with them. A bigger model meant slower inference and higher GPU bills for every single token.

Problem 2: Wasted capacity. Not every input needs every part of the model. Different inputs benefit from different sub-networks, but in a dense model, every input is forced to pass through every parameter, whether it needs them or not.

So, here comes the **Mixture of Experts** to the rescue. It lets us build a model with a huge total number of parameters while only using a small fraction of them for each input. We get the benefits of a massive model without paying the full compute cost.

What Is an Expert?

An **expert** is simply a small neural network. In most modern LLMs, each expert is a small **feed-forward network** - the same kind of feed-forward network we already know from the Transformer architecture.

A Transformer layer in an MoE model can have **8 experts**, or **64 experts**, or even **128 experts** sitting side by side. Each expert is a complete feed-forward network on its own.

Note: The word "expert" can be misleading. An expert is not specially trained on math or physics. The model learns on its own which experts should handle which kinds of inputs during training. In practice, experts often specialize by low-level patterns like punctuation, word types, or token shapes, not by high-level topics. There is no manual labeling.

The Router

Now that we have many experts sitting side by side, the next question is: **how does the model decide which expert to use for each input?**

This is where the **router** comes in. The router is a tiny neural network that looks at each input token and decides which experts should process it. In practice, the router is usually just a **linear layer followed by a softmax**, not a deep network.

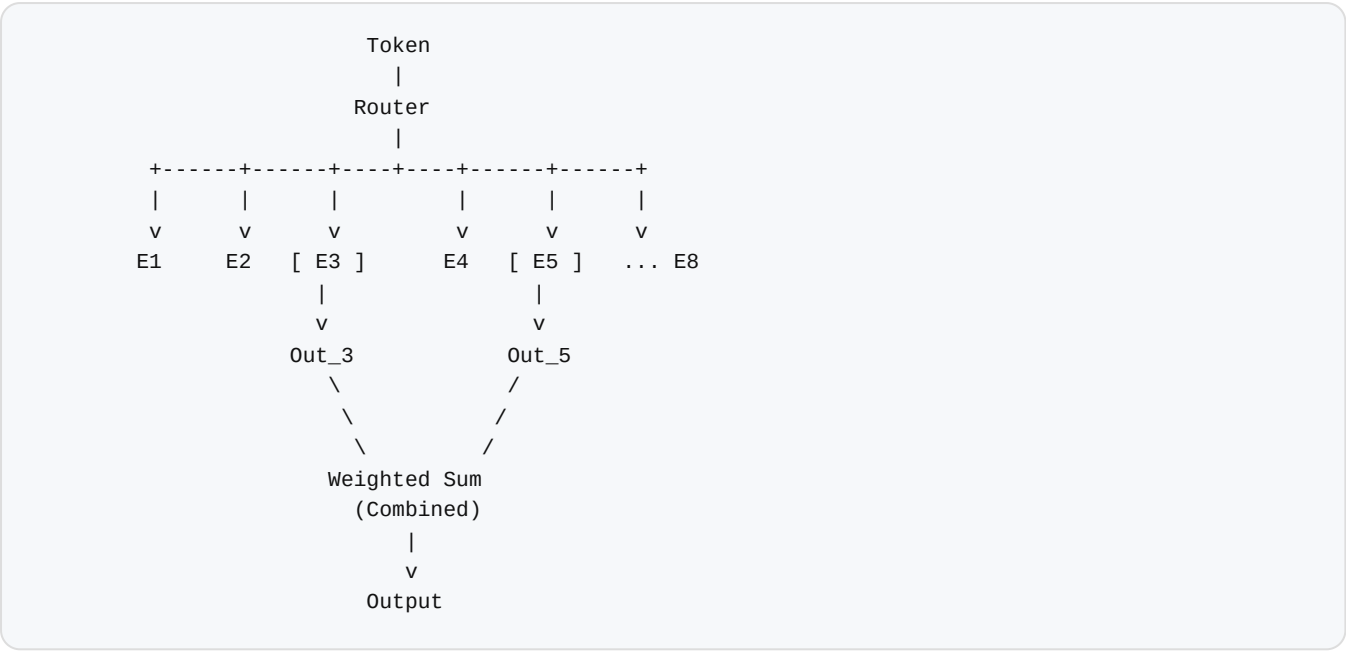
The router works in three simple steps.

First, the router looks at the input token and produces a score for every expert. If there are 8 experts, the router produces 8 scores. The math behind it is simple: **Router(x) = Softmax(x * W_router)**, where `x` is the input token and `W_router` is the router's learned weight matrix. The **softmax** is just a function that turns raw scores into probabilities that add up to 1.

Then, the router picks the **`top-k` experts** with the highest scores. In most MoE models, **`k` = 2**, meaning only the `top-2` experts are picked for each token. Some models like **Switch Transformer** use `k = 1`.

Finally, the token is sent only to those `top-k` experts. The other experts are completely skipped for this token.

Here is a simple visual of the router picking experts. Suppose in this example, the router picks **E3** and **E5**:



Only the picked experts run. The rest stay idle for this token. This is how MoE works at its core.

Where Does MoE Sit Inside a Transformer?

Before we go further, let's quickly recall the structure of a Transformer layer. Each Transformer layer has two main sub-layers:

- A **multi-head attention** sub-layer
- A **feed-forward** sub-layer

In a normal (dense) Transformer, the feed-forward sub-layer is one single big network that every token passes through.

In an MoE Transformer, **the feed-forward sub-layer is replaced with a Mixture of Experts**. The multi-head attention sub-layer stays the same. Only the feed-forward part is replaced.

Here is a simple visual:

```
Input to Transformer Layer
           |
           v
Multi-Head Attention
           |
           v
         Router               (picks top-k experts)
           |
+----+----+----+----+----+----+----+----+
|    |    |    |    |    |    |    |    |
v    v    v    v    v    v    v    v
E1   E2 [E3]   E4 [E5]   E6   E7   E8     (only top-k run)
           |
           v
         Output
```

So MoE does not replace the entire Transformer. It only replaces the feed-forward sub-layer inside each Transformer layer. Everything else stays the same.

Note: Not every layer has to be an MoE layer. Some MoE models alternate between dense layers and MoE layers. This is a design choice.

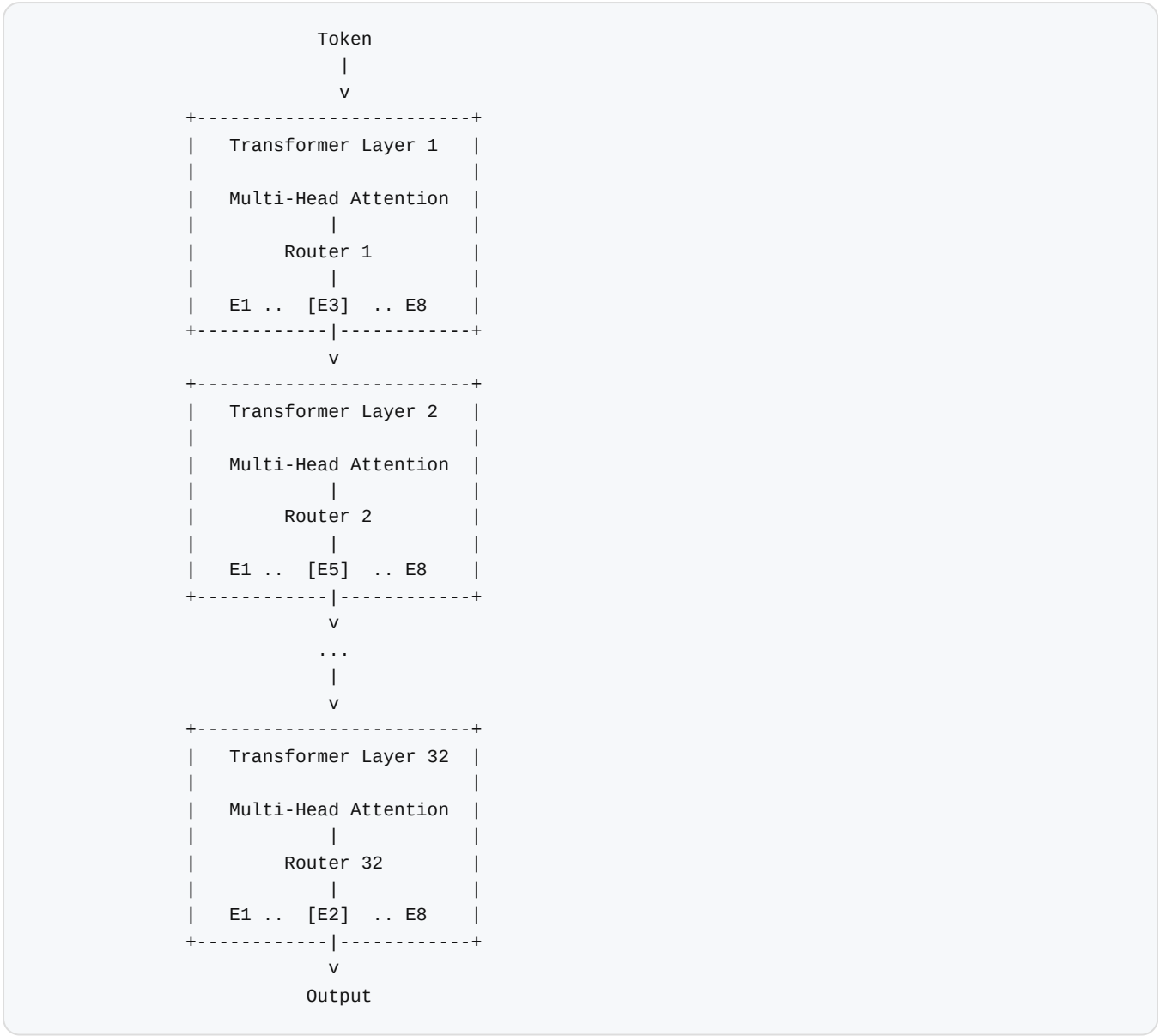
Sparse Activation

This is where MoE becomes powerful. Because only a few experts run for each token, we say the model is **sparsely activated**.

Let's take an example. Suppose we have an MoE model with **8 experts** in each layer, and the router picks the **top 2 experts** for each token. This means only **2 out of 8** experts run for each token. The other 6 stay idle.

Note: Each MoE layer has its own router with its own learned weights. So if a model has 32 MoE layers, it has 32 separate routers - one for each layer. Routers are not shared across layers, because each layer learns its own way of dispatching tokens to its own set of experts.

Here is a simple visual showing one router per MoE layer:



Each MoE layer has its **own** router and its **own** set of experts. The picked experts can be different at every layer.

So even though the model has the total parameters of all 8 experts, the actual compute used per token is only the compute of 2 experts. This is a huge saving.

Active Parameters vs Total Parameters

MoE models are often described as having **"total parameters"** and **"active parameters"**. Total parameters are the full size of the model. Active parameters are the parameters actually used per token.

Let's say each expert has 10 billion parameters. The model has a total of **80 billion parameters** across all 8 experts. But for each token, only **20 billion parameters** are actually used. The model behaves like

a 20-billion-parameter model in terms of compute, but it carries the knowledge of an 80-billion-parameter model.

For example, **Mixtral 8x7B** has **8 experts** and is built on top of a 7-billion-parameter base. The total size is around 47 billion parameters, but only about 13 billion are active per token. The model is fast like a 13-billion-parameter model but smart like a 47-billion-parameter model.

Note: A common confusion is "8 x 7 = 56, so why is Mixtral 47B and not 56B?". The reason is that **only the feed-forward sub-layer is replicated across experts**. The attention layers, embeddings, and normalization layers are **shared** across all experts. So the total parameters are not simply (number of experts) x (expert size).

In simple words: **a big brain with a small bill**.

Load Balancing

Now, here is the catch. What if the router keeps picking the same 2 experts for every token? Then the other 6 experts never learn anything, and the model wastes its capacity.

This is a real problem in MoE training, and it is called the **load imbalance problem**.

To solve this, MoE models use a special technique called **load balancing**. During training, an extra signal called the **auxiliary load balancing loss** (introduced in papers like Switch Transformer and GShard) is added to the main loss. This extra loss pushes the router to spread tokens more evenly across all experts. This way, every expert gets a fair share of tokens to learn from.

Think of it like a manager in an office. If only two employees get all the work and the rest sit idle, the team is unbalanced. A good manager makes sure work is spread fairly across the team. The load balancing loss is exactly that manager for the router.

Without load balancing, MoE models collapse - only a few experts get used and the rest become dead weight. With load balancing, all experts learn useful skills and the model uses its full capacity.

There is also one more practical detail called **expert capacity**. Each expert can only process a fixed number of tokens per batch. If too many tokens get routed to the same expert, the extra tokens are either dropped or sent to the next-best expert. This capacity limit is what makes load balancing critical in real systems.

How Outputs Are Combined

When the router picks the `top-k` experts for a token, each picked expert produces its own output. So how does the model combine these outputs into a single output?

The router does not just pick the `top-k` experts. It also produces **weights** for them - numbers that tell us how much to trust each picked expert. The final output is a **weighted sum** of the outputs from

the picked experts.

In simple words: **final output = weight_1 * Expert_1(input) + weight_2 * Expert_2(input)** (for `top-2` experts).

The weights come from a softmax applied **only over the picked experts' scores** (not all experts), so the weights for the picked experts add up to 1.

Let's walk through a concrete example. Suppose a token enters an MoE layer with 8 experts:

- **Step 1:** The router scores all 8 experts. Say the top-2 are Expert 3 with score `0.7` and Expert 5 with score `0.3`.
- **Step 2:** Only Expert 3 and Expert 5 run. Expert 3 produces an output vector `Out_3`. Expert 5 produces an output vector `Out_5`.
- **Step 3:** The model combines them as: **final output = 0.7 * Out_3 + 0.3 * Out_5**.
- **Step 4:** This final output is passed to the next Transformer layer.

So Expert 3's output is trusted more than Expert 5's output because its router score is higher. This way, the model gets a smooth, blended answer from the top experts instead of a rough one-expert answer.

Advantages of Mixture of Experts

Let's understand why MoE has become so popular:

Massive scale at a fraction of the cost: We can build models with hundreds of billions or even trillions of total parameters while keeping the active parameters per token much smaller. This is the only practical way to push model size to the extreme.

Faster inference: Since only a few experts run per token, the actual compute per token is small. This means faster responses and lower GPU costs in production.

Specialization: Different experts naturally learn to handle different kinds of inputs. The router learns which expert is good at what, without anyone telling it. This gives the model rich, specialized knowledge.

Better scaling: Research has shown that MoE models often reach the same quality as dense models with much less compute during training.

Challenges of Mixture of Experts

MoE is not free. It comes with its own set of challenges:

Memory cost: Even though only a few experts run per token, **all experts must be loaded into memory**. So an 80-billion-parameter MoE model still needs the GPU memory to hold 80 billion

parameters, even if it only uses 20 billion per token.

Load balancing is tricky: Without careful training, the router collapses to a few experts. Engineers need to add load balancing losses and tune them carefully.

Communication cost: In large MoE models, experts often live on different GPUs. Sending tokens to the right GPU and getting the results back adds communication overhead. Serving engines like SGLang are built to spread these experts across many machines and keep this overhead low.

Harder to fine-tune: Fine-tuning MoE models is trickier than fine-tuning dense models because the router behavior can shift during fine-tuning.

Why MoE Powers Many Modern LLMs

Many of today's most powerful LLMs use Mixture of Experts under the hood. **Mixtral**, **DeepSeek-V2**, **DeepSeek-V3**, and several other frontier models are all MoE-based. There are also strong rumors that many of the top closed-source models use MoE internally.

The reason is simple. As models keep getting bigger, dense models hit a wall. Training and serving a dense model with hundreds of billions of parameters is extremely expensive. MoE breaks this wall by letting us grow the total size of the model without growing the per-token compute at the same rate.

In short, MoE is the answer to the question: **how do we keep making models smarter without making them slower and more expensive?**

Summary

Let's recap what we have learned:

- **Mixture of Experts** is an architecture where many small expert networks sit side by side, and a router picks only a few of them for each token.
- **Experts** are small feed-forward networks. They are not manually specialized - the model learns on its own which expert handles what.
- **The router** is a tiny network that scores experts and picks the `top-k` for each token.
- **Sparse activation** means only the picked experts run, which makes MoE fast even when total parameters are huge.
- **MoE replaces the feed-forward sub-layer** inside a Transformer layer. Attention stays the same.
- **Load balancing** is needed to make sure all experts get used during training.
- **Outputs are combined** as a weighted sum based on the router's scores.
- MoE gives us **massive scale at a fraction of the cost**, which is why it powers many of today's most powerful LLMs.

Now we have learned the Mixture of Experts architecture and understood how it works.

Frequently Asked Questions

Does Mixture of Experts reduce the GPU memory needed?

No. Even though only a few experts run per token, all experts must be loaded into memory. So, an 80-billion-parameter MoE model still needs the GPU memory to hold 80 billion parameters, even if it only uses 20 billion per token. MoE saves compute, not memory.

Why is Mixtral 8x7B around 47B parameters and not 56B?

Because only the feed-forward sub-layer is replicated across experts. The attention layers, embeddings, and normalization layers are shared across all experts. So, the total parameters are not simply the number of experts multiplied by the expert size. Only about 13 billion parameters are active per token.

Is each expert trained on a specific topic like math or physics?

No. There is no manual labeling. The model learns on its own which experts should handle which kinds of inputs during training. In practice, experts often specialize by low-level patterns like punctuation, word types, or token shapes, not by high-level topics.

Do all MoE layers share the same router?

No. Each MoE layer has its own router with its own learned weights. If a model has 32 MoE layers, it has 32 separate routers. Each layer learns its own way of dispatching tokens to its own set of experts, so the picked experts can be different at every layer.

What happens if too many tokens are sent to the same expert?

Each expert can only process a fixed number of tokens per batch, which is called expert capacity. If too many tokens get routed to the same expert, the extra tokens are either dropped or sent to the next-best expert. This limit is what makes load balancing critical in real systems.

LESSON 5.3

What is Grouped Query Attention (GQA) and Why Do LLMs Use It?

Lesson 5.3 · ~18 min · Published April 22, 2026 · https://outcomeschool.com/blog/grouped-query-attention



Grouped-Query Attention (GQA) is a strategy where the attention heads are divided into groups, and all the heads within a group share the same Key and Value, while each head still has its own Query.

In this blog, we will learn about Grouped-Query Attention (GQA) and how it differs from Multi-Head Attention (MHA).

We will also learn about Multi-Query Attention (MQA) along the way and see when to use which one.

Grouped-Query Attention is one of the most important optimizations in modern Large Language Models. It allows models to generate text much faster while using less memory, and that too without losing much quality.

Before jumping into Grouped-Query Attention, we must know how Multi-Head Attention works. We have a detailed blog on Transformer Architecture where we have explained Multi-Head Attention as well. I will highly recommend reading it first.

Today, we will cover the following topics:

- The Big Picture
- Quick Recap: Multi-Head Attention (MHA)
- The Problem with Multi-Head Attention
- What is Multi-Query Attention (MQA)?

- What is Grouped-Query Attention (GQA)?
- How Grouped-Query Attention Works
- GQA is a Generalization of MHA and MQA
- GQA vs MHA vs MQA
- Real-World Use Cases
- A Note on Terminology
- Uptraining: Converting MHA to GQA
- Quick Summary

Let's get started.

The Big Picture

Before we go into the details, let's understand the big picture.

In Multi-Head Attention, every head keeps its own Key and Value. This is great for quality, but it makes the KV Cache very large during inference. Grouped-Query Attention keeps the same set of Queries, but it makes groups of heads share one Key and one Value. This shrinks the KV Cache by a lot while keeping the quality very close to Multi-Head Attention. It is one of the main KV Cache Compression techniques, the one that is built into the model design itself.

In simple words:

Grouped-Query Attention = Near Multi-Head Attention quality + Much smaller KV Cache through head grouping.

Quick Recap: Multi-Head Attention (MHA)

In our blog on Transformer Architecture, we have explained Multi-Head Attention in detail. Here is a quick recap.

In Multi-Head Attention, we run the attention mechanism multiple times in parallel. Each parallel run is called a **head**. Each head has its own separate set of weight matrices for **Query (Q)**, **Key (K)**, and **Value (V)**.

If we want to understand the math behind how Q, K, and V actually work in attention, we have a detailed blog on Math Behind Attention: Q, K, V.

Let's say we have a model with 8 heads. This means:

- Head 1 has its own Q_1, K_1, V_1
- Head 2 has its own Q_2, K_2, V_2

- Head 3 has its own Q_3, K_3, V_3
- ... and so on for all 8 heads

Each head looks at the input from a different perspective. The outputs from all heads are concatenated and passed through a final projection to produce the result.

Think of it like a team of 8 detectives investigating a crime scene. Each detective has their own set of clues (K and V) and their own set of questions (Q). Each one investigates independently and finds different insights. When they share their findings, the team gets a complete picture.

This works great during training. But, here is the catch. There is a big problem during **inference** (i.e. when the model is actually generating text for the user). Let's understand this problem.

The Problem with Multi-Head Attention

When a language model generates text, it produces one word at a time. For each new word, the model needs to look back at all the previous words using the attention mechanism.

Note: Technically, language models work with **tokens** (small pieces of text) instead of whole words. But just for the sake of understanding, we will use the word "word" throughout this blog.

Now, here is the important part. To compute attention, the model needs the **Key (K)** and **Value (V)** for every previous word. If the model recomputes K and V for all previous words every time it generates a new word, it would be very slow.

So, the model stores the Key and Value of all previous words in memory. This stored memory is called the **KV Cache**.

The best way to learn this is by taking an example.

Let's say we are generating a sentence and we have already produced 1,000 words. The model has 8 heads. Each head stores its own Key and Value for all 1,000 words.

So, the KV Cache stores:

- 8 sets of Keys (one per head) x 1,000 words = 8,000 Key vectors
- 8 sets of Values (one per head) x 1,000 words = 8,000 Value vectors
- Total: 16,000 vectors in memory

Note: This is per transformer layer. The full KV Cache multiplies this by the number of layers in the model. This is why long-context inference gets memory-hungry so fast.

Now, imagine a large model with 64 heads and a sequence length of 100,000 tokens. The KV Cache becomes extremely large. It takes up a huge amount of GPU memory.

Going back to our detective analogy, each detective (head) maintains their own personal notebook (KV Cache) where they write down clues about every word they have seen. With 64 detectives and 100,000

words, that is 64 separate notebooks, each with 100,000 pages. That is a lot of paper.

This is the problem. The KV Cache in Multi-Head Attention grows very fast and takes up a lot of memory. This makes inference slow and expensive, especially for long sequences and large models.

So, we need a smarter approach that reduces the size of this KV Cache without losing the quality of the model's output. First, let's understand Multi-Query Attention (MQA), which was the first attempt to solve this problem. Then we will learn about Grouped-Query Attention (GQA), which is a better solution.

What is Multi-Query Attention (MQA)?

Multi-Query Attention (MQA) is a strategy where all heads share the same Key and Value, but each head still has its own Query.

Let's decompose the term:

Multi-Query Attention = Multiple Queries + Single shared Key and Value

In Multi-Head Attention, each head has its own Q, K, and V. In Multi-Query Attention, each head has its own Q, but all heads share **one single K** and **one single V**.

Let's say we have 8 heads:

Multi-Head Attention (MHA):

- Head 1: Q_1, K_1, V_1
- Head 2: Q_2, K_2, V_2
- Head 3: Q_3, K_3, V_3
- ... (8 separate K and V sets)

Multi-Query Attention (MQA):

- Head 1: Q_1, K_shared, V_shared
- Head 2: Q_2, K_shared, V_shared
- Head 3: Q_3, K_shared, V_shared
- ... (only 1 K and 1 V set for all heads)

Now, the KV Cache only needs to store 1 set of Keys and 1 set of Values instead of 8. The KV Cache becomes 8 times smaller.

Going back to our detective analogy, in MQA, all 8 detectives share a single notebook of clues. They still ask their own different questions (Q), but they are all looking at the same evidence. This saves a lot of paper (memory), but it limits their ability to find different insights.

The problem with MQA:

MQA reduces memory by a lot, but there is a trade-off. Since all heads share the same Key and Value, the model loses some of its ability to look at the input from different perspectives. The quality of the output can drop, and training can also become unstable.

We need something in between. Something that reduces memory like MQA but maintains quality like MHA.

So, here comes **Grouped-Query Attention** to the rescue.

What is Grouped-Query Attention (GQA)?

Grouped-Query Attention (GQA) is a strategy where heads are divided into groups, and all heads within a group share the same Key and Value, while each head still has its own Query.

Let's decompose the term:

Grouped-Query Attention = Grouped Queries + Shared Key and Value per group

In simple words, instead of giving every head its own K and V (like MHA) or giving all heads a single shared K and V (like MQA), GQA divides the heads into groups and lets each group share one K and V.

The best way to learn this is by taking an example.

Let's say we have 8 heads and we divide them into 2 groups (4 heads per group):

Group 1: Head 1, Head 2, Head 3, Head 4 - share K_group1, V_group1

Group 2: Head 5, Head 6, Head 7, Head 8 - share K_group2, V_group2

So:

- Head 1: Q_1, K_group1, V_group1
- Head 2: Q_2, K_group1, V_group1
- Head 3: Q_3, K_group1, V_group1
- Head 4: Q_4, K_group1, V_group1
- Head 5: Q_5, K_group2, V_group2
- Head 6: Q_6, K_group2, V_group2
- Head 7: Q_7, K_group2, V_group2
- Head 8: Q_8, K_group2, V_group2

Now, instead of storing 8 sets of K and V (like MHA), we only store 2 sets. The KV Cache becomes 4 times smaller.

And instead of using just 1 shared K and V (like MQA), we use 2 sets. So, the model still has some ability to look at the input from different perspectives.

Going back to our detective analogy, in GQA, we divide the 8 detectives into 2 teams. Each team of 4 detectives shares one notebook of clues. This way, we need only 2 notebooks instead of 8 (much less paper), but we still have 2 different sets of clues (better than 1). Each team can discover different things.

GQA is the sweet spot between MHA and MQA. It gives us the memory savings close to MQA while keeping the quality close to MHA.

Note: In GQA, only the Key and Value are shared within a group. The Query is still separate for every head. This is important because the Query is what allows each head to look at the input from a different perspective. By keeping the Queries separate, GQA preserves the diversity of attention patterns. The KV Cache shrinks because we only store the Key and Value during inference, not the Query.

Now, let's understand how GQA actually works step by step.

How Grouped-Query Attention Works

Let's walk through the process step by step.

Step 1: Divide the heads into groups. The number of groups is a setting that we choose before training. Let's say we have 8 heads and 2 groups.

Step 2: Each head computes its own Query (Q) using its own weight matrix. So, all 8 heads have their own separate Queries. This is the same as MHA.

Step 3: Each group computes one shared Key (K) and one shared Value (V) using the group's weight matrices. Group 1 computes K_group1 and V_group1. Group 2 computes K_group2 and V_group2.

Step 4: Each head runs the attention mechanism using its own Query but with the shared Key and Value of its group. Heads 1 to 4 use K_group1 and V_group1. Heads 5 to 8 use K_group2 and V_group2.

Step 5: The outputs from all heads are concatenated and passed through the final projection, just like in MHA.

The result is the same type of output as MHA. But the KV Cache is much smaller because we only store K and V for each group, not for each head.

This is how Grouped-Query Attention works.

Now, let's see something very interesting about GQA.

GQA is a Generalization of MHA and MQA

Here is something beautiful about GQA. It is actually a generalization of both MHA and MQA. Let's understand how.

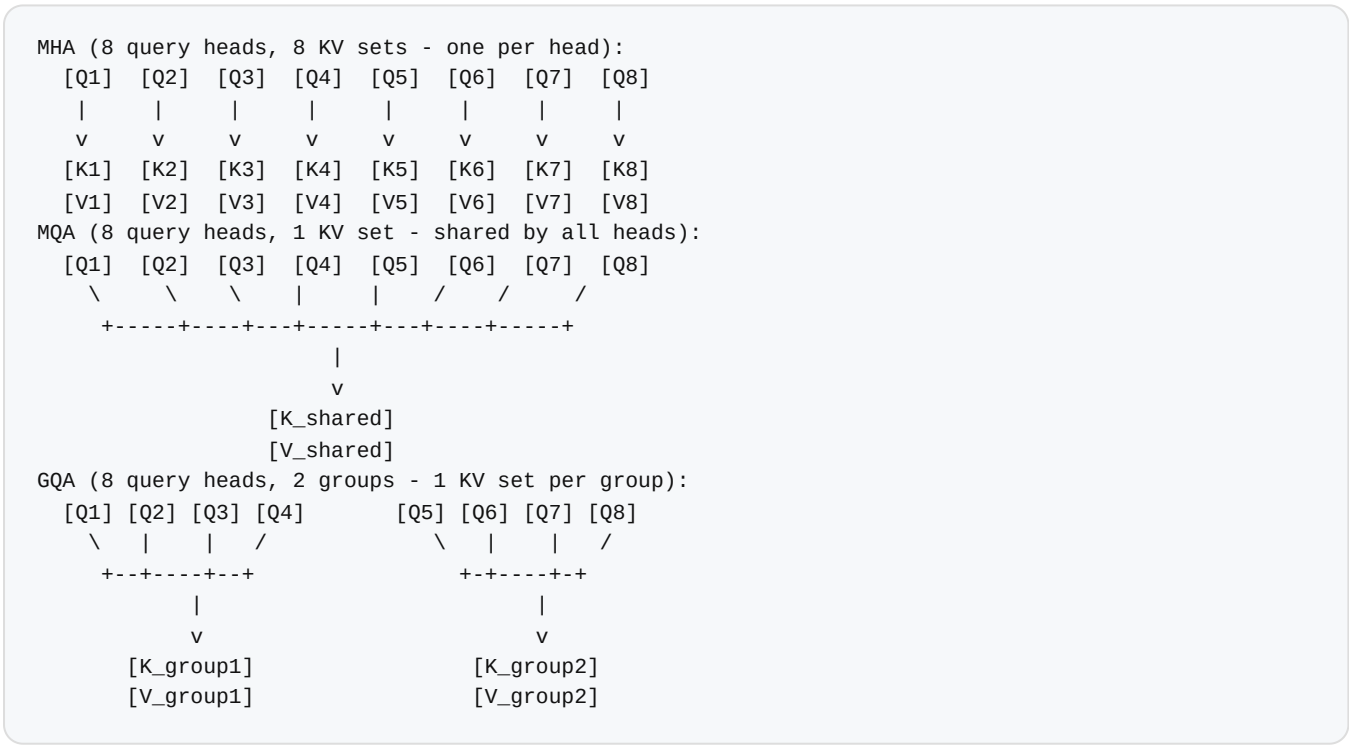
- **When the number of groups = number of heads:** Each group has exactly 1 head. Each head gets its own K and V. This is exactly **Multi-Head Attention (MHA)**.
- **When the number of groups = 1:** All heads are in one group. All heads share the same K and V. This is exactly **Multi-Query Attention (MQA)**.
- **When the number of groups is between 1 and number of heads:** This is **Grouped-Query Attention (GQA)**.

So, MHA and MQA are just special cases of GQA. By changing the number of groups, we can control the trade-off between quality and memory. That's the beauty of Grouped-Query Attention.

Now that we have learned about all three, let's compare them.

GQA vs MHA vs MQA

Before we compare the numbers, let's first see the three architectures visually side by side.



Here, we can see the difference clearly:

- In **MHA**, each Query has its own private Key and Value. 8 Queries, 8 KV sets. Maximum quality, maximum memory.

- In **MQA**, all Queries point to a single shared Key and Value. 8 Queries, 1 KV set. Minimum memory, but reduced diversity.
- In **GQA**, Queries are divided into groups, and each group shares one Key and Value. 8 Queries, 2 KV sets. A balance of both worlds.

Now, let me tabulate the differences between GQA, MHA, and MQA for your better understanding so that you can decide which one to use based on your use case.

Feature	MHA	MQA	GQA
Key-Value sets	One per head	One for all heads	One per group
KV Cache size	Largest	Smallest	In between
Output quality	Best	Can degrade	Close to MHA
Memory usage during inference	Highest	Lowest	In between
Inference speed	Slowest	Fastest	Fast
Example (8 heads, 2 groups)	8 KV sets	1 KV set	2 KV sets

In simple words: MHA gives the best quality but uses the most memory. MQA uses the least memory but can lose quality. GQA is the sweet spot - it uses much less memory than MHA while keeping quality very close to MHA.

Now, let's see some real-world use cases.

Real-World Use Cases

GQA is now used in many popular large language models:

- **LLaMA 2 (34B and 70B):** Uses GQA with 8 KV groups. The 70B model has 64 query heads, so the KV Cache is 8 times smaller compared to standard MHA. (Meta described the 34B in the paper but never publicly released its weights.) The smaller LLaMA 2 models (7B and 13B) still use standard MHA.
- **LLaMA 3:** Uses GQA across all model sizes for faster inference.
- **Mistral 7B:** Uses GQA with 32 query heads and 8 KV groups. It also pairs GQA with Sliding Window Attention so that long inputs stay cheap.

These models use GQA because it allows them to handle longer sequences and serve more users at the same time, and that too while keeping the quality of the output very high.

Serving many users on one GPU also means managing the KV Cache memory carefully at serving time, and we have a detailed blog on vLLM that explains how a serving system handles this.

GQA is especially useful for:

- **Long conversations:** When the model needs to remember thousands of previous words, GQA keeps the KV Cache manageable.
- **Serving many users:** Smaller KV Cache means more users can be served on the same GPU.
- **Edge deployment:** Reduced memory makes it easier to run models on devices with limited resources.

GQA has become a standard technique in modern LLMs. It enables us to have faster inference and lower memory usage without meaningfully sacrificing quality. It makes our life easy.

A Note on Terminology

In the actual code of popular models (like the configurations in HuggingFace or LLaMA), the number of groups is called `num_key_value_heads`. The number of query heads is called `num_attention_heads`.

So, the number of heads per group is simply:

Heads per group = `num_attention_heads` / `num_key_value_heads`

Let's say we read about a model that has 32 query heads and 8 KV heads. This means there are 8 groups, and each group has 4 heads sharing the same Key and Value.

This terminology is good to know because when we read the configuration of a real model, we will see `num_key_value_heads` directly instead of the word "groups".

Uptraining: Converting MHA to GQA

Now, the next big question is: do we always need to train a GQA model from scratch? The answer is no.

The original GQA paper showed that we can take an existing Multi-Head Attention model and convert it into a GQA model very cheaply. This is called **uptraining**.

The process is simple:

Step 1: Take an existing MHA model that has already been trained.

Step 2: For each group, take all the Key weight matrices of the heads in that group and average them. Do the same for the Value weight matrices. This gives us one shared Key and one shared Value per group.

Step 3: Fine-tune the model for a short time. The original paper showed that uptraining with just around 5% of the original pre-training compute is enough to recover quality close to full MHA.

This is one of the main reasons GQA was adopted so quickly. Labs did not have to throw away their existing MHA models or spend huge compute to train new ones. They could just uptrain them into GQA.

Now, we have understood Grouped-Query Attention and how it differs from Multi-Head Attention. We also learned about Multi-Query Attention, how GQA is a generalization of both MHA and MQA, the terminology used in real code, and how existing MHA models can be uptrained into GQA.

Quick Summary

Let's recap what we have learned:

- **Multi-Head Attention (MHA):** Every head has its own Query, Key, and Value. Best quality, but the KV Cache becomes very large during inference.
- **The KV Cache problem:** During text generation, the model stores the Key and Value of every previous word for every head. With many heads and long sequences, this takes a huge amount of GPU memory.
- **Multi-Query Attention (MQA):** All heads share one single Key and one single Value, while each head still has its own Query. The KV Cache becomes very small, but the output quality can drop.
- **Grouped-Query Attention (GQA):** Heads are divided into groups. Each group shares one Key and one Value, while each head still has its own Query. This is the sweet spot between MHA and MQA.
- **GQA is a generalization:** When the number of groups equals the number of heads, GQA becomes MHA. When the number of groups is 1, GQA becomes MQA. Anything in between is GQA.
- **Importance:** GQA gives us memory savings close to MQA while keeping quality close to MHA. It is now used in many popular models like LLaMA 2, LLaMA 3, and Mistral 7B.
- **Uptraining:** We do not need to train a GQA model from scratch. We can take an existing MHA model, average the K and V weights within each group, and fine-tune for a short time to convert it into a GQA model. This is one of the main reasons GQA was adopted so quickly.

Frequently Asked Questions

Why does GQA share the Key and Value but not the Query?

The Query is what allows each head to look at the input from a different perspective, so keeping it separate preserves the diversity of attention patterns. Also, the KV Cache only stores the Key and Value during inference, not the Query. So, sharing only the Key and Value is what shrinks the KV Cache.

Does Grouped-Query Attention reduce model quality?

Only slightly. GQA keeps the output quality very close to Multi-Head Attention while using much less memory. Multi-Query Attention, which shares a single Key and Value across all heads, can lose more quality and can even make training unstable. GQA sits in between, which is why it is called the sweet spot.

Is GQA mainly useful during training or during inference?

GQA is mainly useful during inference. Multi-Head Attention works great during training, but when the model generates text one word at a time, it stores the Key and Value of every previous word in the KV Cache. GQA shrinks this KV Cache, which helps with long conversations, serving many users on the same GPU, and edge deployment.

How do we find the number of GQA groups in a model's configuration?

The number of groups is called `num_key_value_heads`, and the number of query heads is called `num_attention_heads`. Heads per group is `num_attention_heads` divided by `num_key_value_heads`. For example, a model with 32 query heads and 8 KV heads has 8 groups, and each group has 4 heads sharing the same Key and Value.

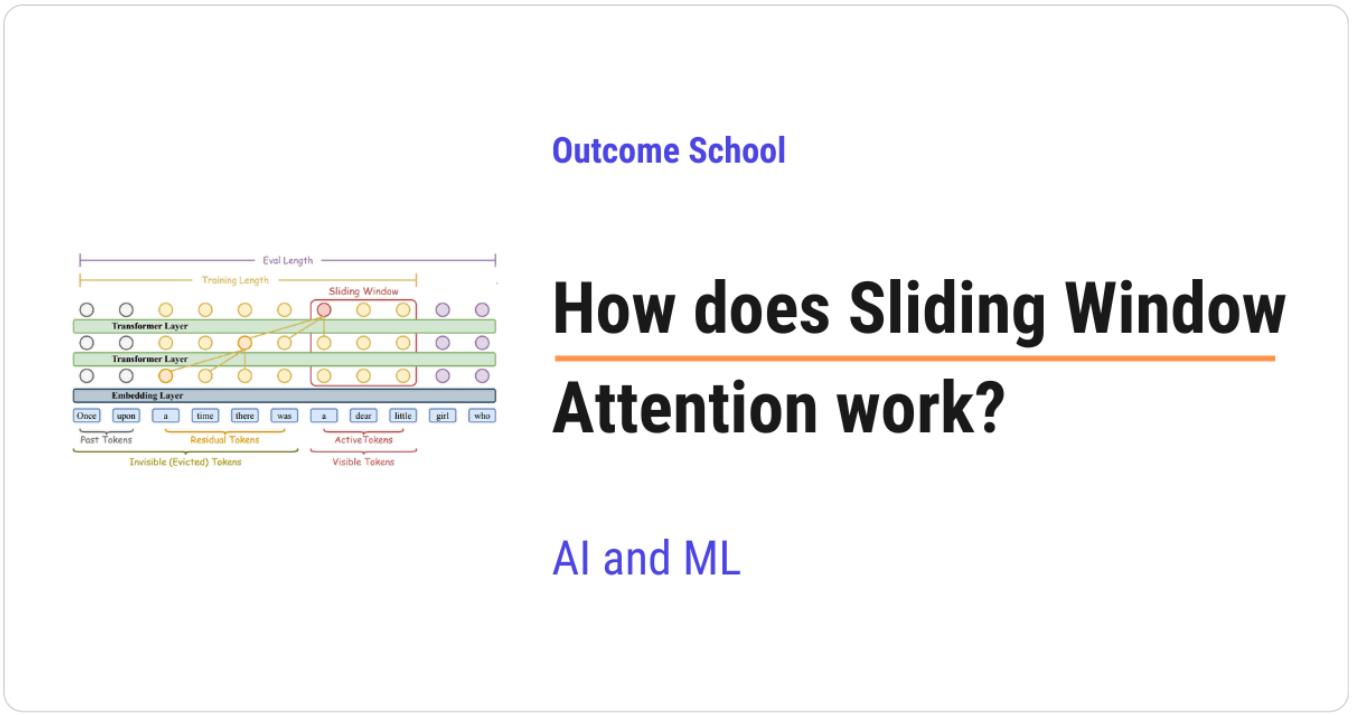
Do all LLaMA 2 models use Grouped-Query Attention?

No. In LLaMA 2, the 34B and 70B models use GQA with 8 KV groups, while the smaller 7B and 13B models still use standard Multi-Head Attention. LLaMA 3 uses GQA across all model sizes, and Mistral 7B also uses GQA.

LESSON 5.4

How does Sliding Window Attention work?

Lesson 5.4 · ~13 min · Published September 5, 2026 · https://outcomeschool.com/blog/how-does-sliding-window-attention-work



Sliding Window Attention is a type of attention where each word looks only at a fixed number of nearby words, instead of looking at all the words. This keeps attention fast and cheap, even for very long text.

In this blog, we will learn about how sliding window attention works. We will also see why normal attention becomes slow and expensive for long text, and how sliding window attention comes to the rescue.

We will cover the following:

- What is attention?
- The problem with normal attention
- What is sliding window attention?
- A simple step-by-step walkthrough
- How information still travels far away
- Comparing normal attention and sliding window attention
- Where sliding window attention is used
- Advantages and trade-offs

Let's get started.

What is attention?

Before we learn about sliding window attention, we must first understand what attention means.

A modern language model, like the one behind a chatbot, reads text and tries to predict the next word. To do this well, it must understand how words connect to each other.

Let's say we read this sentence:

> The cat sat on the mat because it was tired.

Now, the word `it` refers to the `cat`. As humans, we know this without thinking. But a model has to figure it out.

Attention is the mechanism a model uses to look back at earlier words and decide which ones matter for understanding the current word.

In simple words, attention lets every word "pay attention" to other words in the text.

Let's use a simple analogy. Imagine we are in a classroom, and each student is a word. When a new student speaks, that student looks around the room to see who else is relevant to the topic. The new student listens more carefully to some students and ignores others. This act of looking around and deciding who to listen to is attention.

So, attention is how a word gathers information from other words.

Now, we have understood attention. Let's see the problem with it.

The problem with normal attention

In normal attention, every word looks at every other word. This is called **full attention**, because nothing is left out.

Let's understand this with our classroom analogy again.

Suppose there are 10 students in the room. When a new student speaks, that student looks at all 10 students. That is fine. The room is small.

But now, suppose there are 10,000 students in a giant hall. When a new student speaks, that student must still look at all 10,000 students. This becomes very slow and tiring.

This is exactly the problem with normal attention.

Let's put it in numbers. If we have a piece of text with `100` words, then each word looks at `100` words. So the total number of connections is `100 x 100`, which is `10,000`.

Now, suppose the text has `1,000` words. Then the total connections become `1,000 x 1,000`, which is `1,000,000`.

Here, we can notice that when the text gets 10 times longer, the work becomes 100 times bigger.

This is because the cost grows with the square of the number of words. In simple words, doubling the text makes the work four times bigger. Making the text 10 times longer makes the work 100 times bigger.

So, full attention works perfectly for short text. But for very long text, like a full book or a long document, it becomes extremely slow and uses a huge amount of memory.

We needed a solution for that, and sliding window attention was introduced to solve this problem.

What is sliding window attention?

Let's break the term down.

Sliding Window Attention = Sliding + Window + Attention

A **window** is a small range of nearby words. Instead of looking at all words, a word only looks at a fixed number of words around it.

Sliding means this window moves along as we move from one word to the next. Each word has its own window centered on its neighbors.

So, **sliding window attention** is a method where every word only pays attention to a small, fixed number of nearby words, instead of all the words in the text.

Let's go back to our classroom analogy.

Earlier, when a new student spoke, that student looked at all 10,000 students. That was the slow part.

Now, with sliding window attention, the student only looks at the few students sitting nearby. For example, the student looks at the 3 students on the left and the 3 students on the right. That is it.

This small group of nearby students is the window.

When the next student speaks, that student also looks at their own nearby students. So the window slides along the room, one seat at a time.

This is the core idea. A word does not look at everyone. A word looks only at its neighbors.

The size of this window is fixed. We choose a number, for example `4`, `512`, or `4096`. This number is called the **window size**. It tells us how many nearby words each word is allowed to look at.

Now, the work no longer grows with the square of the text length. Instead, it grows in a straight line with the text length. If the text becomes 10 times longer, the work becomes only 10 times bigger, not 100 times bigger.

So, here comes sliding window attention to the rescue.

A simple step-by-step walkthrough

The best way to learn this is by taking an example.

Let's say we have a short sentence with 6 words. We will give each word a number so it is easy to follow:

```
Position:  1    2    3    4    5    6
Word:      I    love  to   learn  new  things
```

Here, we have 6 words placed at positions 1 to 6.

Now, let's choose a small window size. For the sake of understanding, assume that each word can look at itself and the 2 words before it. So the window covers 3 words at a time.

Note: In a chatbot, a word usually looks only at past words and itself, not at future words, because the model reads from left to right and predicts the next word. So in our example, each word looks backward.

We have a detailed blog on Causal Masking in Attention that explains how a model blocks future words step by step.

Let's now go word by word and see which words each one pays attention to.

Position 1 (`I`): There are no words before it. So `I` looks only at itself.

Position 2 (`love`): It looks at itself and one word before it. So `love` looks at `I` and `love`.

Position 3 (`to`): It looks at itself and the 2 words before it. So `to` looks at `I`, `love`, and `to`.

Position 4 (`learn`): The window now slides. So `learn` looks at `love`, `to`, and `learn`. Notice that `I` is now outside the window.

Position 5 (`new`): It looks at `to`, `learn`, and `new`.

Position 6 (`things`): It looks at `learn`, `new`, and `things`.

Here, we can see that the window slides forward by one position each time. Every word looks at a small, fixed group of nearby words.

Let's see it as below:

```
Words:     I    love   to   learn  new  things
Position:  1     2     3     4     5     6
Pos 1:   [ I ]
Pos 2:   [ I    love ]
Pos 3:   [ I    love   to ]
Pos 4:         [ love   to    learn ]
Pos 5:                [ to    learn  new ]
Pos 6:                      [ learn  new   things ]
```

Here, we can see that each row is the window for one word. The box holds the small group of words that the current word looks at. At position 4, the window has moved forward and `I` has dropped out. The box keeps sliding one step to the right, one word at a time.

This is how sliding window attention keeps the work small and steady.

Now, a very natural question comes to mind. If a word only looks at its neighbors, how does it ever connect to words that are far away?

Do not worry, we will learn about that next.

How information still travels far away

This is the most beautiful part of sliding window attention.

At first, it looks like a problem. Word number 100 cannot directly look at word number 1, because word 1 is far outside its window. So how can far away words still connect?

The answer is layers.

A modern model is not a single step. It is made of many **layers** stacked on top of each other. A layer is one full round of attention. The output of one layer becomes the input of the next layer.

We have a detailed blog on Decoding Transformer Architecture that covers how these layers are stacked end to end.

Let's understand this with a simple chain analogy.

Suppose information can only pass between neighbors who are sitting next to each other.

In layer 1, student A whispers to student B, who is sitting right next to A. Student B now knows a little about A.

In layer 2, student B whispers to student C. Now C knows a little about B, and since B already knew about A, C indirectly learns a little about A too.

In layer 3, student C whispers to student D. Now D indirectly learns about A as well.

So, even though no one talks to a far away person directly, information slowly travels across the whole room, one neighbor at a time, through the layers.

Here is the flow:

```
            A     B     C     D
Layer 1:    A --> B
                  B --> C
                        C --> D
Layer 2:    A ......... C  (A reaches C through B)
                  B ......... D  (B reaches D through C)
Layer 3:    A ............... D  (A reaches D through B and C)
```

Here, we can see that in layer 1 each person only talks to a direct neighbor. After layer 2, A has reached C. After layer 3, A has reached D, even though A and D never talked directly. The dotted lines show the indirect path that information takes through the layers.

This is exactly how sliding window attention reaches far away words.

Let's put a simple number to it. Suppose the window allows a word to look 4 words back in each layer. With 1 layer, a word reaches 4 words back. With 2 layers, it reaches about 8 words back. With 10 layers, it reaches about 40 words back.

So, the more layers we stack, the farther the information can travel.

In simple words, one layer connects nearby words directly, and many layers connect far away words indirectly.

The problem is solved.

Comparing normal attention and sliding window attention

Let me tabulate the differences between normal attention and sliding window attention for your better understanding.

Point	Normal attention	Sliding window attention
What each word looks at	Every other word	Only nearby words in a fixed window
Cost as text grows	Grows with the square of the length	Grows in a straight line with the length
Speed on long text	Slow	Fast
Memory on long text	Very high	Low
Far away connections	Direct, in a single layer	Indirect, through many layers
Best for	Short text	Long text

Here, we can see that normal attention is powerful but expensive. Sliding window attention is lighter and scales nicely for long text.

So, now we know where each one fits.

Where sliding window attention is used

Sliding window attention is used in many modern language models that need to handle long inputs.

The reason is simple. When we want a model to read a very long input, full attention becomes too slow and too costly. Sliding window attention lets the model handle that long input quickly while still understanding the overall meaning through its layers.

A real use case is a chatbot that remembers a long conversation. The conversation can grow very long over time. With sliding window attention, the model can keep up without slowing down too much. It also stops the model's memory of the conversation from growing without limit, which is why a sliding window shows up as one of the KV Cache Compression techniques.

Another real use case is reading a long legal document or a long book. The model can process page after page efficiently, because each word only looks at its neighbors directly and reaches far away words through layers.

This is how sliding window attention makes long text practical.

Advantages and trade-offs

Now, let's summarize the good parts and the things we must keep in mind.

Advantages:

- It is much faster on long text, because the work grows in a straight line instead of the square of the length.
- It uses much less memory, which means we can handle longer inputs on the same hardware.
- It still understands long range connections, through the stacking of many layers.
- It is simple to reason about, because each word focuses on a clear, fixed neighborhood.

Trade-offs:

- Far away words are connected only indirectly, through layers. So we need enough layers and a reasonable window size for distant information to travel well.
- A very small window with very few layers can miss some long range connections. We must choose the window size based on our use case.
- For short text, full attention is already fast enough, so sliding window attention gives little benefit there.
- If the window slides far enough that the very first tokens fall out, the quality can collapse, because those tokens act as attention sinks that soak up the leftover attention.

So, sliding window attention is the right choice when the text is long. For short text, normal full attention works perfectly well.

This way we can use sliding window attention to solve the problem of long text in a very simple way. A word looks only at its neighbors, the window slides along, and many layers carry information across the whole text.

Frequently Asked Questions

Can a word look at future words in sliding window attention?

Not in a chatbot. In a chatbot, a word usually looks only at past words and itself, because the model reads from left to right and predicts the next word. So the window covers the current word and a fixed number of words before it, and it slides forward one word at a time.

How far back can a word reach with sliding window attention?

It depends on the window size and the number of layers. If a word can look 4 words back in each layer, it reaches 4 words back with 1 layer, about 8 with 2 layers, and about 40 with 10 layers. The more layers we stack, the farther the information can travel.

Does sliding window attention reduce memory in long conversations?

Yes. Each word only looks at a fixed number of nearby words, so memory use stays low even for long text. It also stops the model's memory of the conversation from growing without limit, which is why a sliding window is one of the KV Cache Compression techniques.

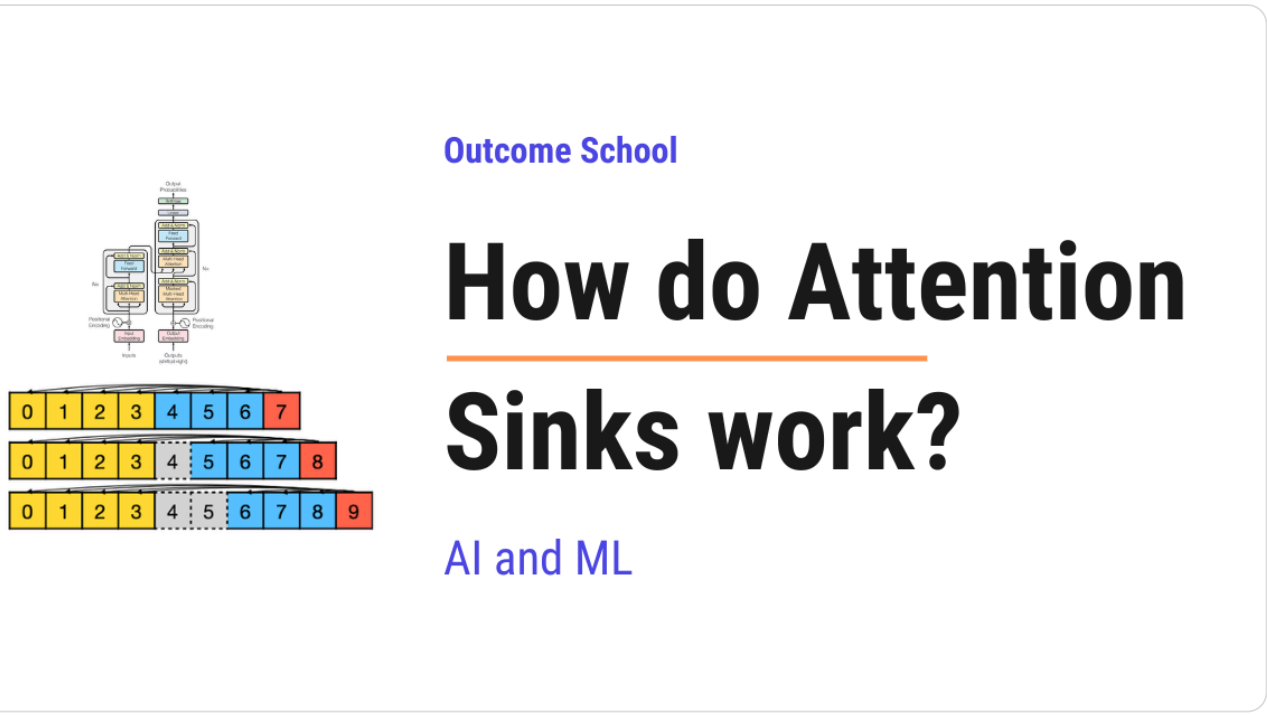
What happens if the first tokens slide out of the window?

The quality can collapse. The very first tokens act as attention sinks that soak up the leftover attention. If the window slides far enough that these tokens fall out, the model loses them and its output can break down.

LESSON 5.5

How do Attention Sinks work?

Lesson 5.5 · ~14 min · Published September 5, 2026 · https://outcomeschool.com/blog/how-do-attention-sinks-work



Attention sinks are the first few tokens of the text that always get a huge share of the attention in a Large Language Model. If we keep these tokens while dropping the old ones, the model can handle very long conversations without breaking.

In this blog, we will learn about how attention sinks work in Large Language Models. We will also see why streaming long conversations becomes a problem, why the first tokens become a sink that the naive fix breaks, and how StreamingLLM uses attention sinks in the real world to keep models running.

We will cover the following:

- What is a Large Language Model?
- What is attention?
- The problem of streaming with long conversations
- The naive fix and why it fails
- What is an attention sink?
- Why the first tokens become a sink
- A step-by-step numeric walkthrough
- The fix in code
- StreamingLLM and modern attention sinks
- Importance of attention sinks

Let's get started.

What is a Large Language Model?

Before jumping into attention sinks, we must know what a Large Language Model is.

A **Large Language Model** is an AI model that reads text and predicts the next word.

In simple words, it is a very good guessing machine. We give it some words, and it guesses the word that comes next. Then it adds that word and guesses again. It keeps doing this, one word at a time.

Let's say we type "The sky is". The model guesses "blue". Now the text becomes "The sky is blue". The model reads all of it again and guesses the next word. This is how a model like ChatGPT writes long answers, one word after another.

Note: The model actually works with small pieces of words called **tokens**. A token can be a full word, a part of a word, or even a single letter. For the sake of understanding, we can think of a token as a word.

This is how a Large Language Model works. Now, let's understand attention.

What is attention?

When the model guesses the next word, it does not look at all the past words equally. Some past words matter more than others.

Attention is the method the model uses to decide which earlier words to focus on while guessing the next word.

In simple words, attention is like focus. When we read a sentence, we focus on the important words and ignore the rest. The model does the same thing.

Let's say we have the sentence "Amit kept his keys on the table, and later he could not find them."

To understand the word "them", the model must focus back on the word "keys". That focusing is attention. The model gives a high score to "keys" and low scores to the other words.

So, attention is just a set of scores. Each past word gets a score that says how important it is for the current word. A high score means high focus. A low score means low focus.

One very important rule: all these scores must add up to 1. This is like sharing one full pie among all the past words. If one word takes a big slice, the others get smaller slices. The model is forced to give away the full pie every single time.

This is how attention works. Now, let's see the problem that creates the need for attention sinks.

The problem of streaming with long conversations

Suppose we are building a chatbot. We want to talk to it for hours and hours without restarting it. The conversation keeps growing longer and longer.

Here is the catch. The model has a limit on how many words it can hold in its memory at once. We call this the **context window**.

In simple words, the context window is the size of the model's short-term memory. If the limit is 4000 words, the model can only keep the last 4000 words in mind.

When the conversation crosses this limit, we have a problem. We must remove some old words to make room for new ones. If we do not remove them, the model runs out of memory and slows down or crashes.

We have a detailed blog on KV Cache in LLMs that explains why this memory keeps growing with every new word.

So, the question is: which old words do we throw away?

The naive fix and why it fails

The most obvious idea is simple. When memory is full, throw away the oldest words and keep only the most recent ones. This is called a **sliding window**.

In simple words, a sliding window keeps a window of recent words and lets the old ones fall off the back, like a moving train window where old scenery slides away.

Let's say the window holds 4 words. The conversation is:

```
Word1 Word2 Word3 Word4 Word5 Word6
```

When Word5 arrives, we drop Word1. When Word6 arrives, we drop Word2. We always keep the latest 4 words.

Let's visualize this as below:

```
Window holds 4 words. New words enter on the right.
Start:     [ Word1 Word2 Word3 Word4 ]
                ^
                first word
Word5 in:  Word1 dropped
           [ Word2 Word3 Word4 Word5 ]
Word6 in:  Word2 dropped
           [ Word3 Word4 Word5 Word6 ]
```

Here, we can see that the oldest word falls off the left every time a new word enters on the right. Word1, the very first word, is the first one to be thrown away.

This sounds perfect. But when researchers tried it, something strange happened. The moment the very first words were dropped, the model broke down. Its answers turned into complete nonsense. The quality fell off a cliff.

This was very surprising. The first words were old chit-chat from hours ago. They had nothing to do with the current topic. So why did dropping them destroy the model?

The issue with this approach is that those first words were secretly doing an important job. Let's see how the next idea explains and solves this issue.

What is an attention sink?

When researchers studied the attention scores, they found a clear pattern. The first few words of the text were always getting a huge share of the attention, no matter what the current word was.

This is the discovery. The model dumps a lot of its attention onto the first few tokens. These tokens are called **attention sinks**.

Let's break the name down for the sake of understanding.

Attention Sink = Attention + Sink

A sink is the drain in a kitchen where extra water flows away. An attention sink is the place where extra attention flows away. The first tokens act like a drain that soaks up the leftover focus.

We can picture the flow as below:



Here, we can see that the model splits its full pie into two parts. The useful focus goes to the important words, and the leftover focus drains away onto the first token, the sink. Nothing is lost, because the drain always has room for the extra.

In simple words, an attention sink is a token that the model uses as a dumping ground for attention it does not need anywhere else.

Now we understand why the sliding window failed. When we dropped the first words, we dropped the drain itself. The leftover attention had nowhere to go, so the whole system broke.

This is the heart of the topic. Now, let's understand why the first tokens become this drain.

Why the first tokens become a sink

Remember the rule from before. All attention scores must add up to 1. The model is forced to give away the full pie on every single word.

But here is the real situation. Many times, the current word does not actually need to focus on anything. It already has enough information by itself.

Let's say the model is processing a simple word that needs no past context. The model still has a full pie of attention to give away. It cannot keep any for itself. It must put the slices somewhere.

So, the model needs a safe drain. It needs a token that it can dump extra attention onto without changing the meaning of anything.

Which token makes the safest drain? The first token. Every word in the text can always see the first token, because the first token is always there from the very beginning. It is visible to everyone. This makes it the perfect common drain.

We have a detailed blog on Causal Masking in Attention that explains how a token is blocked from looking forward, which is what keeps the first token visible to every other token.

So, the model learns a habit. When in doubt, dump the extra attention on the first token. This is why the first tokens become attention sinks.

This is how and why attention sinks form. Now, let's see this with real numbers.

A step-by-step numeric walkthrough

Learning by example is the best way to learn. Let's take a tiny example with 4 words and follow the attention pie.

Assume that we are processing the 4th word. The model has to spread its full pie of attention (a total of 1.0) across the words it can see.

Step 1: The current word truly cares only about Word3. So Word3 deserves a big slice.

Step 2: But the full pie must add up to 1.0. After giving Word3 its fair share, there is leftover attention that nothing really needs.

Step 3: The model dumps that leftover onto Word1, the sink.

The scores look like below:

```
Word1 (the sink): 0.55
Word2:            0.02
Word3:            0.40
Word4 (current):  0.03
Total:            1.00
```

Here, we can see that Word1 grabbed 0.55, which is more than half the pie, even though it is old chit-chat. Word3, the word that actually matters, got 0.40. The rest is tiny leftovers.

Now, suppose we run the naive sliding window and drop Word1 to save memory. The 0.55 slice that used to go to the sink now has nowhere to go. The model is still forced to make the pie add up to 1.0, so it spreads that 0.55 across the remaining words by force.

The new, broken scores:

```
Word2: 0.10
Word3: 0.80
Word4: 0.10
Total: 1.00
```

Here, we can notice that Word3 was wrongly pumped up from 0.40 to 0.80. Every other word also got distorted. The model's focus is now completely wrong, so its next guess turns into nonsense.

This is exactly why dropping the first token destroys the model. The drain is gone, and the dirty water spills everywhere. Now, let's see the fix.

The fix in code

So, here comes the simple fix to the rescue. The idea is easy. When we slide the window to save memory, we must always keep the first few sink tokens and never drop them.

In simple words, keep the drain and slide the rest.

The structure looks like below:

```
Full conversation:
  Word1 Word2 Word3 ...  Word97 Word98 Word99 Word100
  ^^^^^                          ^^^^^^^^^^^^^^^^^^^^
  sinks                          recent window
Broken version (sinks dropped):
  [ Word97 Word98 Word99 Word100 ]
  no drain -> attention spills -> nonsense
Fixed version (sinks kept):
  [ Word1 ] + [ Word97 Word98 Word99 Word100 ]
    keep        keep recent
    drain
  middle is thrown away
```

Here, we can see that the broken version keeps only the recent words and loses the drain. The fixed version keeps the first sink tokens, joins them with the recent window, and throws away only the

middle. The drain stays, so the leftover attention always has a home.

Let's see the code for a normal sliding window first, the broken version:

```
def naive_sliding_window(tokens, window_size):
    # keep only the most recent words
    return tokens[-window_size:]
```

Here, we have kept only the last `window_size` words. This drops the first tokens, which means we drop the sink. This is the version that breaks.

Now, our updated code that keeps the sink:

```
def streaming_window(tokens, num_sinks, window_size):
    # always keep the first few sink tokens
    sinks = tokens[:num_sinks]
    # keep the most recent words
    recent = tokens[-window_size:]
    return sinks + recent
```

Here, we can see what we have done:

- We keep `num_sinks` tokens from the very start. These are the attention sinks, the drain.
- We keep the recent `window_size` tokens, which hold the current topic.
- We join them together and throw away only the middle.

The drain stays in place. The leftover attention always has a home. The model keeps working perfectly even after hours of conversation. The problem is solved.

Note: Researchers found that keeping just the first 4 tokens as sinks is usually enough to keep the model stable.

This is how the fix works in practice. Now, let's learn about the method that made this popular.

StreamingLLM and modern attention sinks

The idea of keeping the sink tokens was introduced in a method called **StreamingLLM**.

In simple words, StreamingLLM is a technique that lets a model keep talking forever without its answers turning into nonsense. It keeps a few sink tokens plus a sliding window of recent tokens, exactly like the code we just saw.

With this trick, a model can handle millions of words of conversation while using only a small, fixed amount of memory. It never crashes and never turns to nonsense. Keeping only the sinks plus a recent window is a form of token eviction, one of the KV Cache Compression techniques that keep this memory small.

There is one more step forward. Some newer models add a special, dedicated sink token on purpose. It is a built-in drain that the model is trained to dump extra attention onto. This is sometimes called a

learnable attention sink, and DeepSeek-V4 uses learnable sinks in exactly this way.

In simple words, instead of borrowing the first real word as a drain, the model gets its own private drain that holds no meaning. This is even cleaner, because the drain is no longer mixed up with a real word.

So, modern models do not rely on luck. They build the drain into the design from day one. This is how attention sinks moved from a surprising discovery to a planned feature.

Importance of attention sinks

Now, we have understood attention sinks fully. Let's see why they are important.

- They let chatbots run long conversations without slowing down or breaking.
- They keep memory usage small and fixed, even for endless chats.
- They explain a strange behavior that confused researchers for a long time.
- They turned a hidden quirk of the model into a useful design tool.

The big lesson is beautiful in its simplicity. The model needed a safe place to throw away the attention it could not use. It quietly chose the first token as that place. Once we understood this, we just had to protect that place, and everything worked.

This way we can use attention sinks to solve the interesting problem of running Large Language Models forever without losing quality.

Frequently Asked Questions

Do attention sink tokens carry important meaning?

Not necessarily. The first tokens can be old chit-chat that has nothing to do with the current topic. They become sinks because every word in the text can always see the first token, so the model learns to use it as a safe drain for the attention it does not need anywhere else.

Why can't the model just give less attention when it needs nothing?

Because all attention scores must add up to 1. The model is forced to give away the full pie of attention on every single word, even when the current word does not need to focus on anything. So, it needs a safe place to put the leftover slices, and the first token becomes that place.

How many sink tokens should we keep in a streaming window?

Keeping just the first 4 tokens as sinks is usually enough to keep the model stable. We join these sink tokens with a sliding window of recent tokens and throw away only the middle. This way, the leftover

attention always has a home.

Does StreamingLLM use more memory as the conversation grows?

No. StreamingLLM keeps a few sink tokens plus a sliding window of recent tokens, so it uses only a small, fixed amount of memory. With this, a model can handle millions of words of conversation without crashing or turning to nonsense.

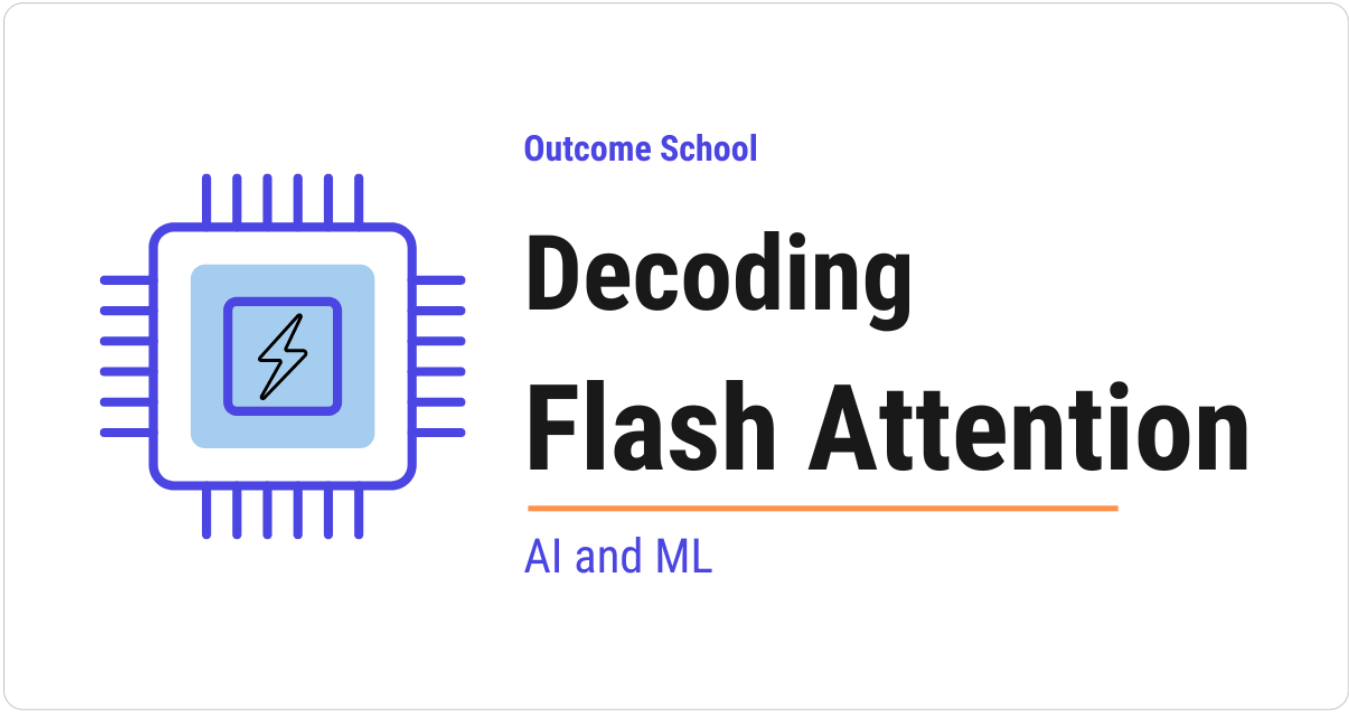
What is the difference between the first-token sink and a learnable attention sink?

A learnable attention sink is a special, dedicated token that the model is trained to dump extra attention onto. Instead of borrowing the first real word as a drain, the model gets its own private drain that holds no meaning. This is even cleaner, because the drain is no longer mixed up with a real word.

LESSON 5.6

What is Flash Attention and Why Is It So Fast?

Lesson 5.6 · ~23 min · Published April 11, 2026 · https://outcomeschool.com/blog/decoding-flash-attention



Flash Attention computes the same attention that Transformers already use, but in a much faster and more memory-efficient way on the GPU. It does not change the math, and it does not change the result.

In this blog, we will learn about Flash Attention by decoding it piece by piece - understanding why standard attention is slow, what makes Flash Attention fast, how it uses GPU memory cleverly, and why it is used in almost every modern Large Language Model (LLM).

When we hear "Flash Attention", it sounds complex. But do not worry. If we break it down into its individual parts, every single piece is simple. Our goal is to decode Flash Attention so clearly that by the end, we will be able to explain it to anyone.

We will cover the following:

- A quick recap of standard attention
- Why standard attention is slow
- How GPU memory actually works (HBM vs SRAM)
- The core idea behind Flash Attention
- Tiling: breaking the work into small blocks
- Online softmax: computing softmax without the full matrix
- Recomputation in the backward pass
- Flash Attention 2

- Flash Attention 3
- Advantages and impact of Flash Attention

Let's get started.

The Big Picture

Before we go into the details, let's understand the big picture.

Flash Attention computes the **same** attention that Transformers already use, but in a much faster and more memory-efficient way. It does not change the math. It does not change the result. It only changes **how** the attention is computed on the GPU so that it runs much faster and uses much less memory.

In simple words: **Flash Attention = Same attention, computed in a much smarter way on the GPU.**

A Quick Recap of Standard Attention

Before jumping into Flash Attention, we must quickly recall how standard attention works.

In a Transformer, each token is converted into three vectors: **Query (Q)**, **Key (K)**, and **Value (V)**. Attention is then computed in three steps.

First, we compute the **attention scores** by multiplying Q with the transpose of K. This gives us a big matrix of scores - one score for every pair of tokens.

Then, we apply **softmax** on these scores so that each row becomes a set of probabilities that add up to 1.

Finally, we multiply these probabilities with V to get the final output.

Attention(Q, K, V) = softmax(Q × K^T / $\sqrt{d_k}$) × V. If we want to go deeper into the math behind Q, K, and V, we can read Math Behind Attention: Q, K, V.

This is the standard attention. It works perfectly. But it has a serious problem when sequences get long.

Why Is Standard Attention Slow?

The issue with standard attention is the **giant intermediate matrix**.

If our sequence has **N** tokens, then the score matrix `Q × K^T` has a shape of **N × N**. Let's put this into perspective with real numbers:

- For an input of **4,000 tokens**, the attention matrix has 4,000 × 4,000 = **16 million** entries.
- For an input of **10,000 tokens**, the attention matrix has 10,000 × 10,000 = **100 million** entries.
- For an input of **100,000 tokens**, the attention matrix has 100,000 × 100,000 = **10 billion** entries.

The memory needed grows with the **square** of the number of tokens. Double the input length, and the memory needed becomes 4 times larger.

There is also a completely different way to attack this cost. We have a detailed blog on Sliding Window Attention that explains how letting each token look only at its neighbors makes the cost grow in a straight line instead of a square.

Here is what happens during standard attention. Notice that the giant N×N matrix is touched **four full times** by the GPU memory:

- **Step 1:** The GPU computes the full N×N matrix and **writes** it to GPU memory.
- **Step 2:** The matrix is **read** back from GPU memory to apply softmax.
- **Step 3:** The softmax result is **written** back to GPU memory.
- **Step 4:** It is **read** again from GPU memory to multiply with V.
- **Step 5:** Finally, the smaller output matrix is written back to GPU memory. This one is not the N×N matrix - it is the much smaller final output.

When we read the "How GPU Memory Actually Works" section below, this will become clearer.

So, the same huge matrix is being moved back and forth many times. This back-and-forth is the real bottleneck. The GPU is not slow at math. The computing cores are fast enough - they spend most of their time **waiting** for data to arrive from slow memory.

This is the problem Flash Attention solves.

How GPU Memory Actually Works

To understand Flash Attention, we must first understand how GPU memory works. A GPU has two main types of memory:

HBM (High Bandwidth Memory): This is the large but **slower** memory on the GPU. It can hold tens of gigabytes, but reading from it and writing to it is slow compared to the GPU's compute speed.

SRAM (Static RAM): This is the small but **very fast** on-chip memory. Reading from SRAM is roughly **10 to 20 times faster** than reading from HBM, but it can only hold a few hundred kilobytes at a time.

Think of HBM as a huge library on another floor, and SRAM as a small desk right in front of you. Walking to the library takes time. Reading on your desk is instant.

The key insight: The GPU is fast, but moving data between HBM and SRAM is slow. So if we keep moving the giant attention matrix between HBM and SRAM, we waste most of our time on data movement instead of math.

Some chips take this idea all the way and keep the whole model inside SRAM so the weights never travel at all, and we have a detailed blog on how an LPU works that explains that design step by step.

Standard attention writes the full N×N attention matrix into HBM. This is the main bottleneck.

The Core Idea Behind Flash Attention

Now, here comes **Flash Attention** into the picture. Flash Attention says:

> Why store the giant N×N attention matrix in HBM at all? Let's compute attention in small blocks that fit inside SRAM, and never write the full matrix to HBM.

In simple words, Flash Attention **avoids creating the giant intermediate matrix** in slow memory. Instead, it processes small chunks at a time, fully inside the fast SRAM, and only writes the final output back to HBM.

This single idea is what makes Flash Attention so much faster than standard attention. No giant matrix in HBM means no expensive back-and-forth data movement.

But doing this is not easy. There are two challenges to solve:

- How do we break the work into small blocks? (**Tiling**)
- How do we compute softmax when we only see one block at a time? (**Online softmax**)

Let's decode each one.

Tiling

Tiling means breaking a large matrix operation into smaller block-sized operations.

Think of it like reading a very long book from the library. The standard approach would be to bring every single page of the book from the library and spread them all over a huge table. This requires a very large table and a lot of walking back and forth to the library. The Flash Attention approach is different. We bring just one small chapter from the library, read it on our small desk, take notes, return it, and then bring the next chapter. We never need a huge table. Our small desk is enough.

Instead of computing $Q \times K^T$ for the entire sequence at once, Flash Attention splits Q, K, and V into small blocks - for example, blocks of 256 tokens each. It then loads one block of Q and one block of K into SRAM, computes the partial attention scores there, and immediately uses those scores with the matching block of V.

Here is a simple visual of tiling:

```
Q split into blocks:        K split into blocks:        V split into blocks:
[Q1] [Q2] [Q3] ... [Qb]     [K1] [K2] [K3] ... [Kb]     [V1] [V2] [V3] ... [Vb]
For each Q block:
   For each K block:
      Load Q_block and K_block into SRAM
      Compute partial scores in SRAM
      Multiply with matching V_block
      Update the running output
```

Because each block is small, both Q_block and K_block fit comfortably inside the fast SRAM. The full N×N matrix is **never** materialized anywhere. The GPU is busy doing math instead of moving data around.

We have a detailed blog on how CUDA Kernels work that explains how GPU threads, blocks, and the fast shared memory work together.

This is the first half of Flash Attention. But there is still one big problem - **softmax**.

The Softmax Problem

Softmax is tricky because it needs to look at the **entire row** to work correctly. If we want to understand softmax in detail, we can watch this video on softmax.

Standard softmax for a row of scores `[s1, s2, s3, ..., sn]` works in two steps.

Step 1: It finds the **maximum** of the row. Let's call it `max`. We do this for **numerical stability** - because the scores can be very large numbers, and `exp` of a very large number can blow up the calculation and produce a **NaN** (Not a Number). By subtracting the max, we keep the numbers small and safe.

Step 2: For each element `si`, it computes `exp(si - max) / sum(exp(sj - max))`. The numerator makes each score positive, and the denominator makes the whole row add up to 1 - so the row becomes a set of probabilities.

For example, for a row `[2, 4, 6]`, the max is 6. We compute `exp(2-6), exp(4-6), exp(6-6)` which is `exp(-4), exp(-2), exp(0)`. Then we divide each by the sum of these three values to get the final probabilities.

The problem is: to find the max and the sum, softmax must see **all the scores in the row** at the same time. But in tiling, we only see one small block of scores at a time. So, how can we compute softmax block by block without ever holding the full row? The answer is **online softmax**.

Online Softmax

Online softmax is a clever trick that lets us compute softmax in pieces. As each new block arrives, we update the running max and the running sum so that the final result is exactly the same as standard softmax - as if we had seen the whole row at once.

Here is the idea in simple words:

- We keep a **running max** of all scores seen so far
- We keep a **running sum** of `exp(score - running_max)` seen so far
- We keep a **running output** of the weighted V values seen so far
- When a new block arrives with a higher max, we **rescale** the previous running sum and running output to match the new max

Let's see this with a small example. Suppose our full row of scores is `[2, 4, 6]` and we see it in two blocks - first `[2, 4]`, then `[6]`.

Step 1: The first block arrives with scores `[2, 4]`. The running max is `4`. The running sum is `exp(2-4) + exp(4-4) = exp(-2) + exp(0) ≈ 0.135 + 1 = 1.135`.

Step 2: The second block arrives with score `[6]`. The new max is now `6`, which is higher than the old max `4`. So we must **rescale** the old running sum to match the new max. We multiply the old sum by `exp(old_max - new_max) = exp(4-6) = exp(-2) ≈ 0.135`. The rescaled old sum becomes `1.135 × 0.135 ≈ 0.153`. Then we add the new block's contribution: `exp(6-6) = 1`. The new running sum is `0.153 + 1 ≈ 1.153`.

If we had done standard softmax on the full row `[2, 4, 6]` at once, the sum would be `exp(2-6) + exp(4-6) + exp(6-6) = exp(-4) + exp(-2) + exp(0) ≈ 0.018 + 0.135 + 1 ≈ 1.153`. Exactly the same result. The running output is rescaled in the same way whenever the max changes.

This way, we never need to hold the full row in memory. After processing all blocks, the running output is exactly the same as the final attention output we would get from standard attention. The softmax problem is solved.

This is the second half of Flash Attention. Combined with tiling, it lets the GPU compute attention without ever storing the giant N×N matrix. It works perfectly.

Putting It All Together

Now, let's put tiling and online softmax together to see how Flash Attention runs end to end.

```
For each block of Q (loaded into SRAM):
    Initialize running_max, running_sum, running_output
    For each block of K and V (loaded into SRAM):
        Compute partial scores = Q_block × K_blockᵀ
        Update running_max
        Rescale running_sum and running_output for the new max
        Add the new contribution from this V_block
    Write the final block of output back to HBM
```

Notice what is happening here. The giant N×N matrix is **never** written to HBM. Only the small Q, K, V blocks move between HBM and SRAM, and only the final output is written back.

Now, a natural question arises - if Flash Attention also moves blocks back and forth many times, why is it faster? The answer is simple: **the total amount of data moved is much smaller.**

Let's take our earlier example of **4,000 tokens** and put real numbers on it. But first, let's quickly understand two terms:

- **Block size:** This is how many tokens we group together in one block. In our example, the block size is **256**, so one Q block contains 256 tokens.
- **The dimension `d`:** This is the number of values used to represent **one token** inside Q, K, and V. So if `d = 128`, each token in Q is a list of 128 numbers. The full Q matrix has `N × d` entries in total. Common values are 64 or 128 in modern LLMs.

Now let's compare how much data each approach moves between HBM and SRAM.

Standard attention creates the full N×N score matrix with **16 million entries** (4,000 × 4,000). Let's also name two intermediate matrices used inside standard attention:

- **S = Scores matrix.** This is the raw output of `Q × Kᵀ` - the attention scores before softmax. Shape: N × N.
- **P = Probabilities matrix.** This is the output of applying softmax on S - each row is now a set of probabilities that add up to 1. Shape: N × N.

So the flow inside standard attention is `S = Q × Kᵀ`, then `P = softmax(S)`, then `O = P × V`.

Now, the N×N matrix gets touched **4 full times** during one attention call:

- **Write S** to HBM → 16M
- **Read S** back to apply softmax → 16M
- **Write P** to HBM → 16M
- **Read P** back to multiply with V → 16M

That gives `4 × 16M = ~64 million entries` just from the N×N matrix moving back and forth. On top of that, there are smaller movements - reading Q, K, V once each, and writing the final output - but these are tiny compared to the N×N matrix and do not change the picture much. So the total data movement for standard attention is roughly **~64 million entries** for one attention call, ignoring any kernel fusion or caching optimizations.

Flash Attention never creates that 16 million entry matrix at all. It works in blocks of 256 tokens. Let's break the numbers down:

- Number of blocks: `4,000 / 256 = 16 blocks` (for each of Q, K, V)
- Size of one block: `256 × 128 = 32,768 entries ≈ 32K`

Flash Attention has two loops - an **outer loop over Q blocks** (16 iterations), and an **inner loop over K and V blocks** (16 iterations inside each outer iteration).

For **each Q block** (one outer iteration):

- Load 1 Q block into SRAM → 32K
- Load all 16 K blocks (one per inner iteration) → `16 × 32K = 512K`
- Load all 16 V blocks (one per inner iteration) → `16 × 32K = 512K`
- **Subtotal per outer iteration ≈ ~1M**

Across all 16 outer iterations: `16 × 1M ≈ 16M entries` of reads. Plus the final output write of `~0.5M` - the output has shape `N × d = 4,000 × 128 = 512K entries`, which is written back to HBM only once at the very end. So the total data movement for Flash Attention is roughly **~17 million entries** - approximate, and the exact number depends on the implementation details and caching.

Here, we can notice that **K and V blocks are re-read once per Q block**. Even with this re-reading, ~17M is still much smaller than ~64M for standard attention because Flash Attention never creates the giant N×N matrix.

Compare the two:

- **Standard attention:** ~64 million entries moved
- **Flash Attention:** ~17 million entries moved

That is roughly **4 times less data movement**, and the gap grows even larger as the sequence length grows. Even though Flash Attention re-reads K and V blocks many times, the total data moved is still far smaller because it never creates the giant N×N matrix. The GPU spends most of its time doing math instead of waiting for data. **That is why Flash Attention is faster.**

Note: We have simplified a few things here to make it easier to understand. In a real Transformer, attention runs in **multiple heads** in parallel, and `d` is actually the **head dimension** (the per-head slice of the full token embedding). We have skipped the multi-head detail because it does not change the core comparison. The exact numbers also depend on the GPU, the block size, and how the implementation is written. The core idea stays the same - Flash Attention moves far less data between HBM and SRAM, and that is why it is faster.

Recomputation in the Backward Pass

There is one more trick that Flash Attention uses during training. During training, we need a **backward pass** to compute gradients. The backward pass normally needs the attention matrix that was computed in the forward pass.

But Flash Attention never stored that matrix in HBM. So how does the backward pass work?

The answer is simple: **Flash Attention recomputes the attention scores in SRAM during the backward pass**. It does not start completely from scratch - during the forward pass, it saves a small amount of **softmax statistics** for each row (just the running max and the running sum), which is only O(N) extra memory. In the backward pass, it uses these saved statistics together with Q, K, and V to quickly rebuild the attention scores inside fast SRAM, without ever writing the giant matrix back to HBM. This sounds wasteful, but it is actually faster overall because recomputing in fast SRAM is cheaper than reading a giant matrix from slow HBM.

This trade-off - **less memory traffic, more compute** - is one of the key reasons Flash Attention is so fast on modern GPUs. Modern GPUs have far more compute than memory bandwidth, so trading memory for compute is a great deal.

Flash Attention 2

After Flash Attention was introduced, the researchers improved it further and released **Flash Attention 2**. Flash Attention 2 uses the same core idea - tiling and avoiding the full attention matrix in HBM. But it reduces wasted work even further by reorganizing the computation in three ways.

Change 1 - Fewer non-matmul operations. In Flash Attention 1, the running output is rescaled **every** time a new K, V block arrives. This rescaling is a non-matmul operation, and non-matmul operations run much slower on modern GPUs than matrix multiplications - because modern GPUs have special **tensor cores** that are extremely fast at matmuls but not at anything else. Flash Attention 2 delays this rescaling and does it only **once at the very end** of each Q block. This removes a lot of slow work from the inner loop.

Change 2 - Better parallelism across the sequence. Flash Attention 1 only parallelized work across **batch size × number of heads**, which works well for short sequences with large batches but leaves the GPU underused for long sequences with small batches. Flash Attention 2 adds one more axis of parallelism - it also **parallelizes across Q blocks** along the sequence dimension. Different Q blocks can be processed **completely in parallel** by different GPU workers, with no need to share results between them. This lets Flash Attention 2 use the full GPU much more efficiently, especially for long sequences.

Change 3 - Better work split inside each worker. Inside each GPU worker, Flash Attention 2 also splits the work differently among its threads, which reduces extra reads and writes to fast memory.

Here is a simple way to remember the difference. **Flash Attention 1 is mostly about cutting the trips to slow HBM.** It took the trips from very high down to a small number. **Flash Attention 2 is mostly about keeping the GPU fully busy doing fast matmul work** once those trips are already cut. It makes sure the tensor cores are not sitting idle while the GPU does slow rescaling or waits for other workers. Because of these improvements, Flash Attention 2 is roughly **2x faster** than Flash Attention 1, and that too with the same mathematical output.

Note: The pseudocode we saw in the "Putting It All Together" section above already follows the Flash Attention 2 style - Q on the outside, K and V on the inside.

Flash Attention 3

After Flash Attention 2, the researchers also released **Flash Attention 3**, which is built specifically for the latest NVIDIA Hopper GPUs (like the H100 and H200). It uses the same core ideas but takes advantage of new hardware features to go even faster.

The key trick in Flash Attention 3 is **asynchronous copy**. In Flash Attention 2, the GPU still had to wait a little bit for each new K, V block to arrive from HBM before it could compute on it. Flash Attention 3 uses the H100's **Tensor Memory Accelerator (TMA)** - a special hardware unit that moves data in the background while the tensor cores keep doing math at the same time. This overlap means the GPU is almost never waiting - it is always computing.

Since Flash Attention 3 needs the latest Hopper GPUs, **Flash Attention 2 is still the most widely used version today** - most developers are still on A100s or consumer cards like the RTX 3090 and RTX 4090. Flash Attention 3 is strictly for the H100 and H200 generation, where it is the fastest option available. That's the beauty of it - as GPUs keep evolving, Flash Attention keeps getting better along with them.

Advantages of Flash Attention

Let's understand why Flash Attention has become the default in almost every modern LLM:

Much faster: Flash Attention is typically **2x to 4x faster** than standard attention for long sequences. The longer the sequence, the bigger the speedup.

Much less memory: Memory usage drops from **$O(N^2)$** to **$O(N)$**. This means we can train models on much longer sequences using the same GPU.

Exactly the same result: Flash Attention is not an approximation. The output is **exactly the same** as standard attention. There is no quality loss.

Enables long context windows: Before Flash Attention, processing very long inputs (like a 100,000-word document) was extremely expensive because the attention matrix was too large to fit in

memory. Flash Attention makes this possible by never creating the full matrix. This is one of the key reasons modern LLMs can handle very long conversations and documents.

Drop-in replacement: Flash Attention works with the same Q, K, V interface as standard attention. We can replace standard attention with Flash Attention without changing anything else in the model.

Used everywhere in modern AI: Today, almost every serious LLM training run and inference engine uses Flash Attention or one of its successors (Flash Attention 2, Flash Attention 3). It has quietly become one of the most important pieces of infrastructure in modern AI. One such inference engine is vLLM, and we have a detailed blog on how vLLM works that explains LLM serving in depth. NVIDIA ships the same fused-kernel idea inside TensorRT-LLM as its custom attention kernels.

I personally believe that Flash Attention is one of the most elegant pieces of systems engineering in modern AI. It takes the same math, keeps the same result, and still makes everything run much faster - that too on the same hardware. It makes our life easier.

Quick Summary

Let's recap what we have decoded:

- **Flash Attention** computes the exact same attention as standard attention, just in a much smarter way on the GPU.
- **Standard attention is slow** because it creates a giant N×N matrix in slow HBM and moves it back and forth many times.
- **GPUs have two memory types:** HBM (large but slow) and SRAM (small but very fast). Data movement between them is the real bottleneck.
- **Tiling** breaks Q, K, and V into small blocks that fit in SRAM, so the giant matrix is never materialized.
- **Online softmax** lets us compute softmax block by block by tracking a running max, running sum, and running output.
- **Recomputation** in the backward pass trades a little extra compute for huge memory savings.
- **The result is the same** as standard attention - just much faster and lighter.
- **Flash Attention 2** keeps the GPU's tensor cores fully busy by doing fewer non-matmul operations and parallelizing across Q blocks, making it roughly 2x faster than Flash Attention 1.
- **Flash Attention 3** is built for the latest NVIDIA Hopper GPUs and goes even faster by using new hardware features, while Flash Attention 2 remains the most widely used version today.
- Flash Attention is now the default in almost every modern LLM and is what makes today's long context windows possible.

This is how Flash Attention quietly runs inside almost every modern LLM today.

Frequently Asked Questions

Is Flash Attention an approximation of standard attention?

No. Flash Attention is not an approximation. It computes the exact same attention and gives exactly the same output as standard attention, so there is no quality loss. It only changes how the attention is computed on the GPU, which makes it faster and more memory-efficient.

Is Flash Attention faster because it does less math?

No. Flash Attention is faster because it moves far less data between slow HBM and fast SRAM. It even does extra compute in the backward pass by recomputing attention scores. Modern GPUs have far more compute than memory bandwidth, so trading memory traffic for compute is a great deal.

Do we need to change the model to use Flash Attention?

No. Flash Attention works with the same Q, K, V interface as standard attention. We can replace standard attention with Flash Attention without changing anything else in the model. That is why it is a drop-in replacement.

Which version of Flash Attention should we use?

It depends on the GPU. Flash Attention 3 is strictly for NVIDIA Hopper GPUs like the H100 and H200, where it is the fastest option available. Since most developers are still on A100s or consumer cards like the RTX 3090 and RTX 4090, Flash Attention 2 is still the most widely used version today.

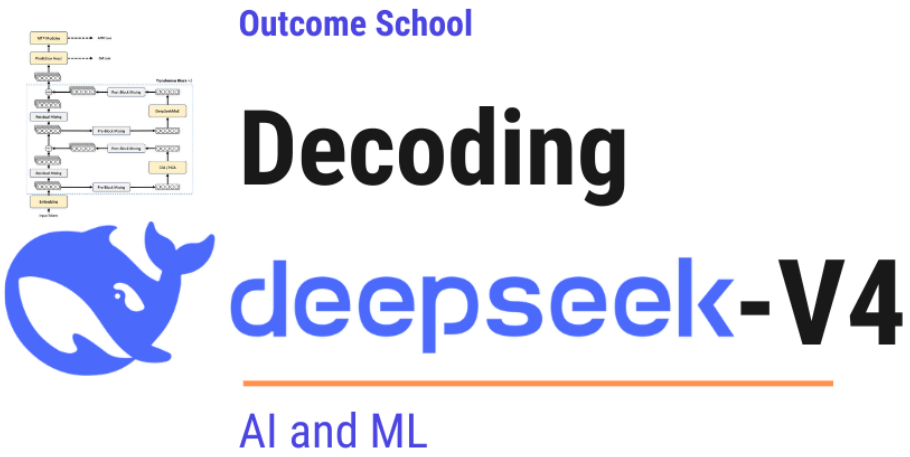
Is Flash Attention the same as sliding window attention?

No. Flash Attention computes the full attention, just in a smarter way on the GPU. Sliding window attention is a completely different way to attack the cost, where each token looks only at its neighbors. That makes the cost grow in a straight line instead of a square.

LESSON 5.7

What is DeepSeek-V4 and How Does It Work? Architecture Explained

Lesson 5.7 · ~20 min · Published April 24, 2026 · https://outcomeschool.com/blog/decoding-deepseek-v4



DeepSeek-V4 is a family of open Mixture-of-Experts language models that supports a one-million-token context, at roughly a tenth of the cost of DeepSeek-V3.2.

In this blog, we will learn about DeepSeek-V4, the new family of open Mixture-of-Experts language models that natively supports a one-million-token context with dramatically lower inference cost.

DeepSeek-V4 makes one-million-token context roughly a tenth as expensive as it was in DeepSeek-V3.2. It introduces a new attention design, a new way of doing residual connections, a new optimizer, and a new post-training pipeline. We will decode each of these one by one.

We will cover the following:

- The Big Picture
- Two Models: DeepSeek-V4-Pro and DeepSeek-V4-Flash
- Hybrid Attention with CSA and HCA
- Manifold-Constrained Hyper-Connections (mHC)
- Muon Optimizer
- FP4 Quantization-Aware Training
- Pre-Training
- Post-Training: Specialist Training and On-Policy Distillation

- Reasoning Modes
- Putting It All Together
- Quick Summary

Let's get started.

The Big Picture

Before we go into the details, let's understand the big picture.

DeepSeek-V4 is built to solve one core problem: **make ultra-long context cheap**. Modern reasoning models and AI Agents need to handle very long inputs - long documents, long tool histories, long agent trajectories. The standard attention mechanism in a Transformer becomes very expensive as the input gets longer. DeepSeek-V4 redesigns attention so that even a one-million-token input remains practical.

In simple words:

DeepSeek-V4 = DeepSeek-V3 + Hybrid Compressed Attention + Better Residual Connections + Muon Optimizer + On-Policy Distillation.

Let's put this into perspective with real numbers. At a one-million-token context:

- DeepSeek-V4-Pro uses only **27%** of the per-token compute of DeepSeek-V3.2.
- DeepSeek-V4-Pro uses only **10%** of the KV cache size of DeepSeek-V3.2.
- DeepSeek-V4-Flash uses only **10%** of the per-token compute and **7%** of the KV cache size of DeepSeek-V3.2.

This is a massive efficiency gain. Now, let's decode each piece.

Two Models: DeepSeek-V4-Pro and DeepSeek-V4-Flash

DeepSeek-V4 ships as two models. Both are **Mixture-of-Experts (MoE)** models. In an MoE model, the feed-forward layer is split into many small expert networks, and a router picks only a few experts per token. This is how the total parameter count can be huge while the per-token compute stays small.

Let me tabulate the differences between the two models for your better understanding so that you can decide which one to use based on your use case.

Model	Total Parameters	Activated Parameters	Transformer Layers	Context Length	Pre-training Tokens
DeepSeek-V4-Flash	284B	13B	43	1M	32T
DeepSeek-V4-Pro	1.6T	49B	61	1M	33T

Here, "activated" means the parameters that actually run for each input token. In an MoE model, only a small fraction of the experts are active per token, so activated parameters are much smaller than total parameters.

Both models support a context of **one million tokens** natively. Both are trained on more than 32 trillion tokens.

DeepSeek-V4-Pro is the larger, more capable model. DeepSeek-V4-Flash is the smaller, faster, more cost-efficient model.

Hybrid Attention with CSA and HCA

This is the most important change in DeepSeek-V4. Let's understand the problem first.

The Problem. Standard attention compares every query token with every other token. Suppose we have N tokens in the input. The cost grows like N x N. At one million tokens, this becomes one trillion comparisons across the full sequence. The KV cache, which is the memory of past keys and values, also grows linearly with N. Both of these become a serious bottleneck for long context.

We have a detailed blog on KV Cache in LLMs that explains what KV cache is and why it matters, and another on KV Cache Compression that covers the standard ways to shrink it.

The Solution. Here comes the hybrid attention design to the rescue. DeepSeek-V4 introduces two new attention types and uses both of them in an interleaved manner across layers:

- **CSA (Compressed Sparse Attention)** - lighter compression, but uses sparse selection.
- **HCA (Heavily Compressed Attention)** - heavier compression, but keeps attention dense.

Let's decode the names first.

CSA = Compressed + Sparse + Attention.

HCA = Heavily + Compressed + Attention.

The best way to learn this is by taking an example. Let's say we are reading a huge library of one million pages and we want to answer a question. Standard attention is like reading every single page from start to end before answering. It works, but it is painfully slow.

CSA and HCA both fix this, but in different ways. We will use this library example throughout to understand both. Let's decode each one.

CSA: Compressed Sparse Attention

CSA does two things to make attention cheap.

Step 1: Compress. Instead of keeping the KV cache for every single token, CSA combines every `m` consecutive tokens into one compressed entry using learned softmax weights. Just for the sake of understanding, let's say `m = 4`. So every 4 tokens get combined into 1 entry in the KV cache. The KV cache shrinks by a factor of `m`.

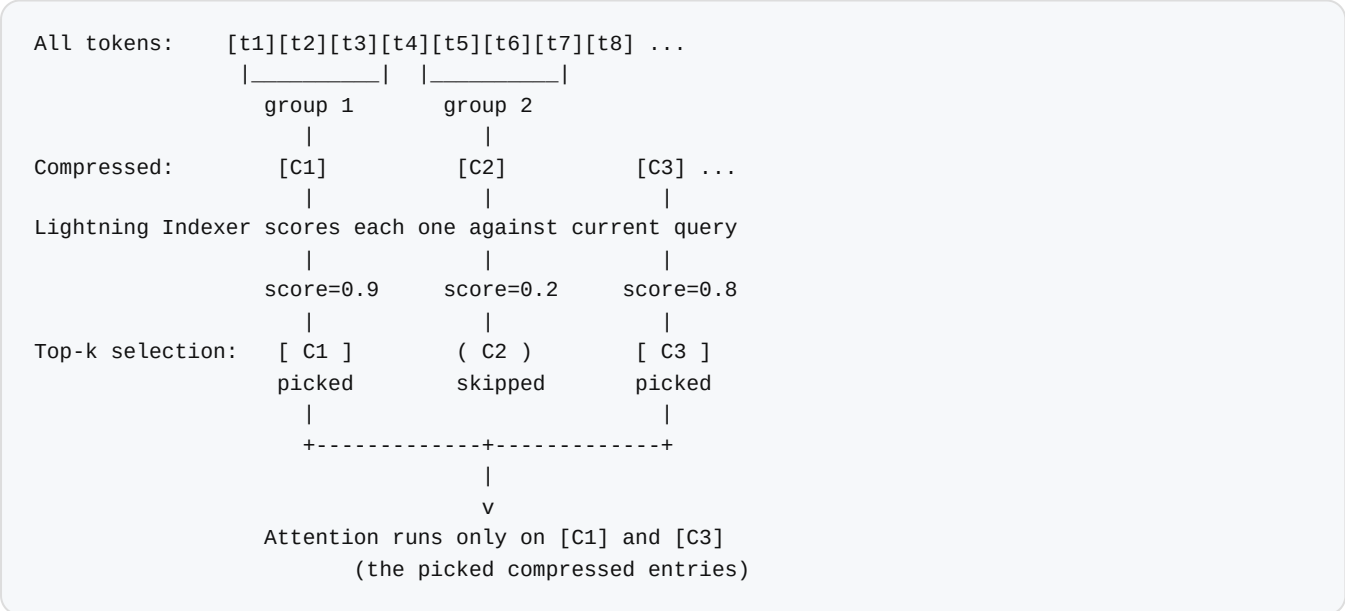
In our library example, this is like grouping every 4 pages into one summary card. Instead of reading 1 million pages, we now have only 250K summary cards to look at.

Step 2: Select. After compression, CSA still does not look at every summary card. It uses a small **Lightning Indexer** to score each compressed entry against the current query, and then picks only the top-k most relevant ones. The Lightning Indexer runs in FP4 precision to keep this scoring step cheap.

In our library example, this is like a quick scanner who flips through the 250K summary cards, scores each one for relevance, and hands us only the top few cards. We then read only those few cards in detail. Most of the library is skipped completely.

For a deeper background on Q, K, V in attention, we have a detailed blog on Math Behind Attention: Q, K, V.

Here is a simple ASCII picture:



So CSA first compresses many tokens into one, and then picks only the few compressed entries that look most relevant. This is how CSA brings the cost down. Both compute and memory are saved.

HCA: Heavily Compressed Attention

This was all about CSA. Now, let's move to HCA.

HCA makes a different trade-off. It compresses much more aggressively, and because the compressed sequence is already tiny, dense attention on it is cheap. No need for sparse selection.

HCA uses a much larger group size `m'`, much bigger than `m` in CSA. For DeepSeek-V4, `m'` is set to 128. So instead of combining 4 tokens into one entry, HCA combines 128 tokens into one entry. The KV cache shrinks even more dramatically.

In our library example, this is like grouping every 128 pages into one summary card. Now we have only about 7,800 summary cards in total. Since the number of cards is already very small, we just read all of them, every time. There is no need to pick the top few.

So CSA and HCA are two different design points with the same goal - cheap long-context attention. CSA does mild compression plus sparse top-k selection. HCA does aggressive compression plus dense attention. The result for both: very low compute and very low memory.

This was all about HCA. Now, let's see how the actual model uses both.

In DeepSeek-V4, the layers are interleaved. For DeepSeek-V4-Flash, the first two layers use pure sliding window attention, and the rest alternate between CSA and HCA. For DeepSeek-V4-Pro, the first two layers use HCA, and the rest alternate between CSA and HCA. This way, different layers handle long context in slightly different ways, and together they cover both fine-grained selection (CSA) and broad coverage (HCA).

Sliding Window Branch

Now, a natural question arises - what if compression blurs out important details from very recent tokens?

The answer is the **sliding window branch**. Both CSA and HCA also keep a small extra set of recent uncompressed KV entries. For DeepSeek-V4, this window size is 128 tokens.

In our library example, this is like always keeping the last 128 pages we just read on our desk in full. Compression is used for the older pages, but the most recent pages are right there in front of us, uncompressed and ready to read. This makes sure that local fine-grained dependencies are not lost.

Attention Sink

One more small but clever trick. In the core attention of both CSA and HCA, DeepSeek-V4 adds a small set of learnable **sink logits** to the denominator of the softmax. This allows each attention head to have attention scores that sum to *less than* 1.

Why does this help? In standard attention, the scores are forced to sum to exactly 1. This means every query token must distribute all of its attention somewhere, even when nothing in the context is

actually relevant. Attention sinks let the model say "none of these are important right now" and spend less attention overall.

DeepSeek-V4 also applies RMSNorm on the queries and compressed KV entries just before the core attention step. This is what keeps attention logits stable and prevents them from exploding.

Note: Put together, CSA, HCA, Sliding Window, and Attention Sink make one trade-off possible: **less memory, less compute, with no major loss in quality**. That is what makes one-million-token context practical.

Manifold-Constrained Hyper-Connections (mHC)

This was all about attention. Now, let's move to the residual connections inside the Transformer.

Let's decode the name first.

mHC = Manifold + Constrained + Hyper + Connections.

In a normal Transformer, every layer adds its output to a "residual stream" - a single shared signal that flows from the bottom of the model to the top. Hyper-Connections (HC) widen this residual stream by a factor of `n_hc` (set to 4 in DeepSeek-V4), turning the single stream into multiple parallel streams. Three learned matrices (A, B, C) mix these streams at every layer, giving the model more flexibility in how information flows.

The issue with plain HC is that it becomes numerically unstable when we stack many layers. The signal can blow up or collapse. Training breaks down.

So, here comes mHC to the rescue. mHC fixes the problem by **constraining** the residual mapping matrix to lie on a special manifold called the Birkhoff polytope - the set of doubly stochastic matrices. A doubly stochastic matrix is a matrix whose every row sums to 1 and every column sums to 1. This simple constraint bounds the largest stretch factor (the spectral norm) of the matrix at 1, so the signal cannot blow up as it passes through dozens of layers.

The result is that deeper stacks train stably, and the model expressivity is preserved. It works perfectly.

Muon Optimizer

This was all about mHC. Now, let's move to the optimizer.

DeepSeek-V4 swaps the optimizer for most of its parameters from AdamW to **Muon**.

Muon = An optimizer that uses the full gradient matrix and orthogonalizes the update direction.

Why does this matter? Most optimizers like AdamW treat each weight independently. Muon, on the other hand, looks at the full gradient matrix and applies an orthogonalization step before doing the weight update. This orthogonalization is done using Newton-Schulz iterations. It normalizes all the

singular values of the update matrix to be close to 1, so no single direction in the update dominates. The result is faster convergence and more stable training.

A few important points:

- AdamW is still used for the embedding module, the prediction head, all RMSNorm modules, and the static parts of mHC.
- Muon is used for everything else.
- The DeepSeek team uses a hybrid Newton-Schulz iteration with carefully chosen coefficients for stable orthogonalization.

This gives us the best of both worlds. Muon brings faster convergence where it matters, and AdamW stays where it is already proven to work well.

FP4 Quantization-Aware Training

Now, let's move to how DeepSeek-V4 makes inference faster and cheaper. The answer is **FP4 Quantization-Aware Training**.

Let's decode this.

- **FP4** stands for floating point with only 4 bits per number. Compared to FP8 or BF16, FP4 uses much less memory and bandwidth. It makes our life easier on the hardware side.
- **Quantization-Aware Training (QAT)** means the model is trained while simulating the lower-precision number format, so the weights adapt to the precision loss during training itself. Without QAT, we would squeeze the weights down only at deployment time, and the model would lose quality. With QAT, the model learns to live with low precision.

DeepSeek-V4 applies FP4 to two key places:

- **MoE expert weights**, which take up most of the GPU memory.
- The **Query-Key (QK) path of the CSA indexer**, where the lightning indexer computes scores in FP4.

There is one more clever detail here. FP8 (E4M3) has 2 more exponent bits than FP4 (E2M1), which gives it a much wider dynamic range. Because of this, the FP4 block scale factors can be fully absorbed into the FP8 format during dequantization. This means the FP4-to-FP8 dequantization is **lossless**, and the entire existing FP8 training pipeline can be reused without any modification.

The end result is a 2x speedup on the top-k selector while preserving a 99.7% recall of KV entries. There is no major quality loss.

Pre-Training

Now that we have learned about the architecture and optimizer, it's time to learn about pre-training.

Data. The pre-training corpus has more than **32 trillion tokens**. It includes mathematical content, code, web pages, long documents, scientific papers, and multilingual data. The team puts a special emphasis on long-document curation since the model is built for long context.

Sequence length scheduling. Training does not start at one-million tokens directly. First, the model trains at a sequence length of 4K. Then, the length is extended to 16K. After that, to 64K. Finally, to 1M. Sparse attention is also turned on gradually. The model first warms up with dense attention for the first 1T tokens, and sparse attention is introduced at 64K and kept on for the rest of the training.

Mitigating Training Instability. Training trillion-parameter MoE models is hard. Loss spikes happen. But here is the catch - simple rollbacks do not prevent the next spike from happening. The DeepSeek team found two practical tricks that fix this:

- **Anticipatory Routing.** The MoE routing network at step `t` is computed using slightly older parameters `θ_{t-Δt}`, while the rest of the backbone uses the current parameters `θ_t`. This asymmetry breaks the feedback loop where bad routing causes outliers, which then cause worse routing.
- **SwiGLU Clamping.** The output of the SwiGLU activation is clamped to a fixed range. This prevents extreme values from appearing and destabilizing training.

With these two tricks in place, training stays stable all the way through.

Post-Training: Specialist Training and On-Policy Distillation

Till now, we have learned about pre-training. Now, it's time to learn about post-training.

Post-training is where DeepSeek-V4 takes a different path from many other models. The pipeline has two stages. Let's decode each one.

Stage 1: Specialist Training

For each target domain like mathematics, coding, agent use, instruction following, and etc., the DeepSeek team trains a separate **expert model**. Each expert model is built in two steps.

First, the base model goes through Supervised Fine-Tuning (SFT) on high-quality, domain-specific data. This establishes the foundational capability for that domain. Then, Reinforcement Learning is applied using **GRPO (Group Relative Policy Optimization)**, guided by domain-specific reward signals. This sharpens the model for the specific success criteria of that domain.

After this stage, the team has multiple specialist expert models - one per domain, each trained for varying reasoning efforts - each excelling in its own field.

This was all about Stage 1. Now, let's move to Stage 2.

Stage 2: On-Policy Distillation (OPD)

Now, a natural question arises - how do we combine all these expert models into one final unified model?

The answer is **On-Policy Distillation (OPD)**.

In simple words:

On-Policy Distillation = The student model generates its own outputs, and learns to match the probability distribution of the right teacher expert for each task.

Let's understand what makes OPD different from regular distillation.

In regular distillation, the student learns from data that the teacher produced. But there is one catch - the student never sees the kind of outputs it actually produces at inference time. This creates a gap between training and real behavior.

In OPD, the student samples its own trajectories. This is the "on-policy" part. The student writes its own drafts, and the teacher corrects each token by providing the full vocabulary distribution it would have used. The student is pulled towards the teacher's distribution using **reverse KL loss**. Reverse KL is mode-seeking - it pushes the student to concentrate probability mass where the teacher's mass is concentrated, rather than smearing out across all teacher modes. This gives sharper, more decisive outputs.

Also, different teachers are weighted based on the task. For a math task, the math expert teaches more. For a coding task, the coding expert teaches more. This way, all the specialist knowledge gets merged into one single unified student.

This step replaces the traditional mixed RL stage that DeepSeek-V3.2 used. The team found that OPD gives more stable and faithful knowledge transfer than weight merging or mixed RL, which is why it became the core consolidation step in the DeepSeek-V4 post-training pipeline.

Reasoning Modes

This was all about post-training. Now, let's move to how DeepSeek-V4 actually serves users at inference time.

DeepSeek-V4 supports three reasoning modes that the user can pick depending on how hard the task is. Let me tabulate the differences between them for your better understanding so that you can decide which one to use based on your use case.

Reasoning Mode	When to Use	How it Works
Non-think	Routine daily tasks, quick answers	No `<think>` block, direct response
Think High	Complex problem-solving, planning	`<think>` block with conscious reasoning
Think Max	Pushing reasoning to its limit	Long `<think>` block with a special "absolute maximum" instruction injected

For **Think Max** mode, the injected system prompt essentially tells the model: *Reasoning effort is set to absolute maximum with no shortcuts permitted. Be very thorough, decompose the problem to its root cause, rigorously stress-test your logic against all edge cases and adversarial scenarios, and write out every intermediate step, considered alternative, and rejected hypothesis explicitly.* This instruction stretches the reasoning as far as it can go.

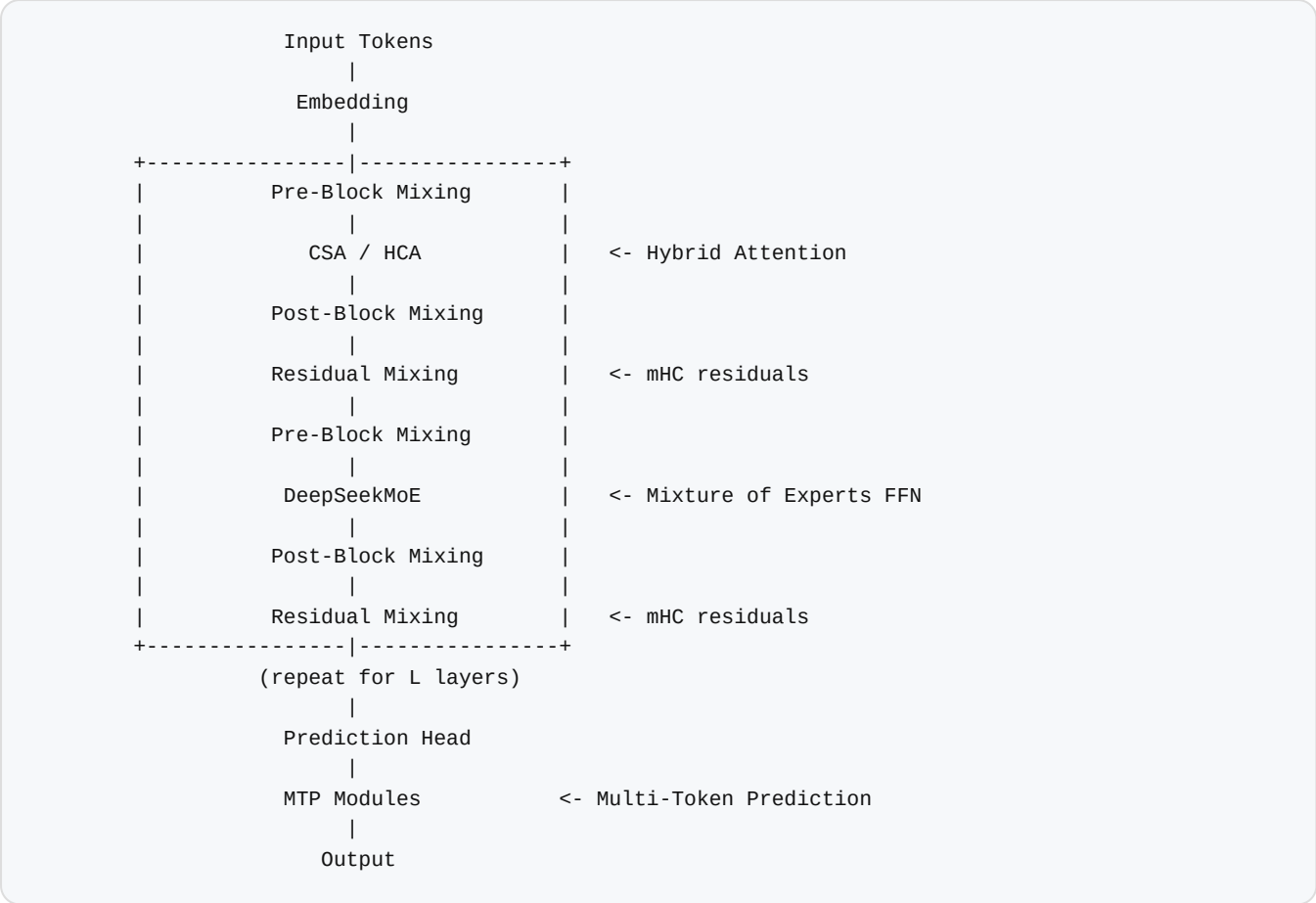
This way, the model can be cheap when the question is easy, and slow but powerful when the question is hard. We get to control the trade-off based on our use case.

Putting It All Together

Now, let's put everything together to see how DeepSeek-V4 runs end to end.

In one sentence: a token enters the embedding layer, flows through many Transformer blocks (each one alternating CSA/HCA attention with a DeepSeekMoE feed-forward layer, both wrapped by mHC residual connections), and exits through the prediction head and MTP modules.

Here is the high-level architecture of one Transformer block in DeepSeek-V4:



At the top of the stack, a prediction head and **MTP (Multi-Token Prediction)** modules produce the output. MTP allows the model to predict more than one token at a time during training, which improves data efficiency, and at inference it can also serve as a built-in drafter for Speculative Decoding. We have a detailed blog on Medusa that explains how extra prediction heads like these speed up generation without a separate draft model.

This is how DeepSeek-V4 runs end to end - combining strong capability with one-million-token efficiency.

Quick Summary

Let's recap what we have decoded:

- **DeepSeek-V4** is a family of open MoE language models built for ultra-long context. Two variants: **V4-Pro (1.6T total, 49B active)** and **V4-Flash (284B total, 13B active)**, both natively supporting **1M tokens**.
- **Hybrid Attention** combines **CSA (Compressed Sparse Attention)** and **HCA (Heavily Compressed Attention)** in an interleaved manner. CSA compresses tokens into groups and then picks top-k. HCA compresses even more but keeps attention dense. A small sliding window keeps recent tokens uncompressed, and learnable attention sinks keep attention logits stable.

- **mHC (Manifold-Constrained Hyper-Connections)** widens the residual stream by `n_hc=4` and constrains the mixing matrix to the Birkhoff polytope (doubly stochastic matrices, rows and columns summing to 1), which bounds the spectral norm at 1 for stable deep training.
- **Muon Optimizer** orthogonalizes the gradient update matrix using Newton-Schulz iterations, normalizing all singular values close to 1 for balanced, faster convergence. AdamW is still used for embeddings and a few other modules.
- **FP4 Quantization-Aware Training** is applied to MoE expert weights and the CSA indexer, with a lossless FP4-to-FP8 dequantization that reuses the existing FP8 pipeline unchanged.
- **Pre-training** uses 32T+ tokens with a sequence length schedule that grows from 4K to 1M, plus **Anticipatory Routing** and **SwiGLU Clamping** to fix loss spikes.
- **Post-training** has two stages: **Specialist Training** (SFT + GRPO per domain) followed by **On-Policy Distillation (OPD)** that merges all specialists into one unified student model.
- **Three reasoning modes** - Non-think, Think High, Think Max - let the user trade off cost and depth.
- **Net result:** at 1M-token context, DeepSeek-V4-Pro uses only 27% of the FLOPs and 10% of the KV cache of DeepSeek-V3.2. DeepSeek-V4-Flash pushes this further to 10% FLOPs and 7% KV cache.

This is how the DeepSeek-V4 series ushers in a new era of million-length contexts for open large language models.

Frequently Asked Questions

Should we use DeepSeek-V4-Pro or DeepSeek-V4-Flash?

We should pick DeepSeek-V4-Pro when we need the larger, more capable model, and DeepSeek-V4-Flash when we need a smaller, faster, more cost-efficient model. Pro has 1.6T total and 49B activated parameters, while Flash has 284B total and 13B activated. Both support a context of one million tokens.

How is DeepSeek-V4 different from DeepSeek-V3?

DeepSeek-V4 is DeepSeek-V3 plus Hybrid Compressed Attention, better residual connections, the Muon optimizer, and On-Policy Distillation. At a one-million-token context, DeepSeek-V4-Pro uses only 27% of the per-token compute and 10% of the KV cache size of DeepSeek-V3.2. It also replaces the mixed RL stage of DeepSeek-V3.2 with On-Policy Distillation.

Does FP4 quantization reduce the quality of DeepSeek-V4?

Not in a major way, since there is no major quality loss. The model is trained with Quantization-Aware Training, so the weights adapt to the lower precision during training itself. FP4 gives a 2x speedup on the top-k selector while preserving a 99.7% recall of KV entries.

Does DeepSeek-V4 use the Muon optimizer for every parameter?

No. Muon is used for most parameters, but AdamW is still used for the embedding module, the prediction head, all RMSNorm modules, and the static parts of mHC. This gives the best of both worlds. Muon brings faster convergence where it matters, and AdamW stays where it is already proven to work well.

Why is on-policy distillation better than regular distillation?

On-policy distillation closes the gap between training and real behavior. In regular distillation, the student learns from data the teacher produced, so it never sees the kind of outputs it actually produces at inference time. In on-policy distillation, the student samples its own outputs, and the teacher corrects each token.

Personal offline study copy · Content © Outcome School / Amit Shekhar